TECHNICAL DOCUMENTATION

RewaCiti

Codebase Walkthrough

A complete, up-to-date guide to the RewaCiti React + TypeScript frontend — every source file, service, store, page, and data flow.

Updated November 2026 · 76 source files · React 18 · TypeScript · Vite · Tailwind v4 · Zustand

Every section shows real source code and then explains what it does. Read top to bottom: each file is presented in roughly the order the app runs it.

Area	Tools
Frontend	React 18 · TypeScript 5.9 · Vite (rolldown-vite)
Routing	React Router 7 (`react-router`)
State	Zustand 5
Styling	Tailwind CSS v4 · `tw-animate-css`
HTTP	Axios
Maps	Leaflet · React Leaflet · OpenStreetMap
Payments	Paystack (`@paystack/inline-js`, `react-paystack`)
UI	Radix UI · Lucide React · React Icons · Sonner
SEO	React Helmet Async
APIs	SabiFlow Real Estate public API · SabiFlow customer API

> This document replaces the previous walkthrough. It covers all 76 files under `src/` plus the build and deployment configuration.

Contents

1. How the app boots
2. Types — the vocabulary of the whole app
3. Shared layer: API client, payments, utils, theme, images
4. Auth: store, services, and pages
5. Properties: services and the store
6. Properties: the listing page
7. Properties: the detail page
8. Properties: shared components
9. Properties: modals and forms
10. Properties: request, co-rent, and list pages
11. Blog, comments, and FAQ
12. Layout: Navbar and Footer
13. Marketing and legal pages
14. Build, config, and deployment
15. End-to-end data flows

---

1. How the app boots

`src/app/main.tsx` — entry point (11 lines)

Purpose: Mounts React into `#root` and loads the global stylesheets.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './styles/App.css'
import App from './App.tsx'
import "leaflet/dist/leaflet.css";

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

How it works

- `index.html` contains a single `<div id="root">`; React replaces its contents.
- `App.css` is the Tailwind entry and is imported first so utilities are available.
- `leaflet/dist/leaflet.css` is imported globally so map styles exist before any map renders.
- `StrictMode` double-invokes effects in development, which helps catch cleanup bugs.

`src/app/App.tsx` — providers, theme sync, wishlist hydration (108 lines)

Purpose: Sets up routing, the toast system, theme synchronisation, and wishlist hydration.

```
function App() {
  const theme = useThemeStore((s) => s.theme);
  const { token, isAuthenticated } = useAuthStore();

  useEffect(() => {
    if (theme === "dark") {
      document.documentElement.classList.add("dark");
    } else {
      document.documentElement.classList.remove("dark");
    }
  }, [theme]);

  useEffect(() => {
    if (isAuthenticated && token) {
      void usePropertyStore.getState().fetchWishlist();
    }
  }, [isAuthenticated, token]);
  return (
    <HelmetProvider>
      <BrowserRouter>
        <Toaster
          theme={theme as "light" | "dark"}
          closeButton
          position="top-right"
          toastOptions={{
            className: "font-sans",
            style: { borderRadius: "1rem" },
            classNames: {
              toast: "rounded-2xl bg-white dark:bg-[#1A1A1A] text-gray-900 dark:text-white border-gray-200 dark:bord
er-white/10 shadow-lg border",
              // ...
            }
          }}
        />
        <Routes>
          {/* ... */}
        </Routes>
      </BrowserRouter>
    </HelmetProvider>
```

```
  )
}
```

Routes in the current app

Path	Page
`/`	`Home`
`/auth/login`	`Login`
`/auth/verify-email`	`VerifyEmail`
`/auth/forgot-password`	`ForgotPassword`
`/auth/reset-password`	`ResetPassword`
`/auth/google`	`GoogleCallback`
`/auth/profile`	`AccountPage`
`/auth/profile/settings`	`Profile`
`/AllComments`	`AllCommentsPage`
`/AllFAQs`	`AllFAQPage`
`/About`	`About`
`/properties`	`PropertiesPage`
`/properties/:slug`	`PropertyDetailsPage`
`/request-property`	`RequestPropertyPage`
`/co-rent`	`CoRentPage`
`/list-property`	`ListPropertyPage`
`/property-feature-icons`	`PropertyFeatureIconsPage`
`/Service`	`Service`
`/Contact`	`Contact`
`/Studentarea`	Redirects to `/properties`
`/terms`	`Terms`
`/privacy-policy`	`PrivacyPolicy`
`/blog`	`BlogPage`
`/blog/:slug`	`BlogPostDetailsPage`
`*`	`NotFound`

How it works

- Provider order matters: `HelmetProvider` → `BrowserRouter` → `Toaster` + `Routes`.
- The `dark` class on `<html>` is the single switch for the whole app. Tailwind v4 declares `@custom-variant dark (&:is(.dark *))`, so every `dark:` utility only applies while that class is present.
- Wishlist hydration calls `fetchWishlist()` directly on the store (`getState()`) because this is not a component.
- There is no route guard. `/auth/profile` is publicly routable; pages check the auth store themselves.
- The toaster is themed with the same brand colours as the rest of the app.

---

2. Types — the vocabulary of the whole app

The app uses two type modules:

- `src/types/realEstate.ts` — the shapes the SabiFlow Real Estate API actually returns.
- `src/types/index.ts` — the app-facing `Property` shape and the Zustand store contracts.

`toProperty()` (see §5) translates the wire shape into the app shape.

`src/types/realEstate.ts` — the wire shape (246 lines)

Purpose: Describes the SabiFlow Real Estate public API exactly, including listings, filters, forms, visits, payments, and deals.

Key exports

- `ListingPurpose`, `PricePeriod`, `SharingType`, `VisitMode`, `RequestKind` — string unions.
- `PublicListing` — the list/detail payload.
- `PublicListingDetail` — adds `description`, `agent`, `visitFee`, and `visitFeeBreakdown`.
- `PropertyFilters`, `PropertyLocations`, `PropertyType` — filter and location metadata.
- `PublicForms` — company form fields and rules.
- `VisitSlot`, `Booking`, `Deal`, `PurchaseResult`, `PaymentStart`, `Sent<T>`, `Paged<T>`.

```
export interface PublicListing {
  id: string;
  slug: string;
  url: string | null;
  reference: string | null;
  title: string;
  type: string | null;
  typeId: string | null;
  purpose: ListingPurpose;
  price: number;
  totalPrice: number;
  pricing: {
    base: number;
    charges: ListingCharge[];
    serviceCharge: number;
    total: number;
  };
  currency: string;
  pricePeriod: PricePeriod;
  term: { value: number; unit: "days" | "months" | "years" } | null;
  status: "published" | "reserved";
  featured: boolean;
  location: {
    state?: string;
    city?: string;
    area?: string;
    nearby: string | null;
    lat: number | null;
    lng: number | null;
  };
  // ...
}
```

How it works

- Field names follow the API exactly, so services can return API payloads without renaming.
- `PublicListingDetail` is detected at runtime with `"visitFee" in listing` inside `toProperty`.
- `Paged<T>` is the standard `{ data, total, page, limit }` envelope.
- `Sent<T>` is the standard write response: `{ message, data, test? }`.
- `PublicForms` is what makes the site adaptive: it decides which visit modes, payment rules, report reasons, and request types are available.

`src/types/index.ts` — the app shape and store contracts (212 lines)

Purpose: Defines the `Property` the pages render, plus the interfaces for every Zustand store.

```
export interface Property {
  id: string;
  slug: string;
  reference?: string;
  name: string;
  img: string;
  images: string[];
  description: string;
  bedrooms: number;
  bathrooms: number;
  category: string;
  purpose: ListingPurpose;
  listingType: string;
  duration: string;
  sharing: SharingType;
  places: { total: number; left: number };
  reserved: boolean;
  pricing: {
    base: number;
    charges: ListingCharge[];
    serviceCharge: number;
    total: number;
  };
  agentId: string | null;
  agent?: PublicListingDetail["agent"];
  location: {
    state: string;
    city: string;
    area: string;
    nearby: string;
  };
  geo_location: { lat: number; lng: number };
  keyFeatures: string[];
  additionalFeatures: string[];
  billsAndCharges: string[];
  additionalBillsAndCharges: string[];
  rules: string[];
  additionalRules: string[];
  specialNotes: string[];
  specifications: { label: string; value: string }[];
  videoUrl?: string;
  visitationfee: number | null;
}
```

Store contracts

- `PropertyStore` — listings, featured listings, categories, forms, locations, related listings, the current property, pagination, wishlist, and all fetch/toggle actions.
- `CommentStore`, `FAQStore`, `BlogStore` — static-content stores with three/six-item pagination.
- `Category`, `Comment`, `FAQ`, `BlogPost`, `ValueItem`, `Step`, `TeamMember`, `AchievementItem` — content shapes.

How it works

- `PropertyStore` keeps both `page` (0-based, for the UI) and `apiPage` (1-based, for the `?page=` query).
- `filters` is a loose `Record<string, string | number | boolean | undefined>` because filter names map straight to API query parameters.
- `visitationfee` is `number | null`: `null` means the detail page has not loaded the fee yet, `0` means a free visit.

---

3. Shared layer: API client, payments, utils, theme, images

`src/shared/lib/realEstateApi.ts` — the shared Axios client (83 lines)

Purpose: One Axios instance for the entire Real Estate public API. It attaches the public key, attaches the customer token when signed in, and handles expired sessions.

```
export const realEstateApi = axios.create({
  baseURL: `${API_URL}/real-estate/public/v1`,
  timeout: 20_000,
});

realEstateApi.interceptors.request.use((config) => {
  config.headers["x-api-key"] = import.meta.env.VITE_SABIFLOW_PUBLIC_KEY;
  const token = useAuthStore.getState().token;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

How it works

- `API_URL` comes from `VITE_API_URL`, falling back to `https://api.sabiflow.com/api`.
- Every request carries `x-api-key` (`VITE_SABIFLOW_PUBLIC_KEY`), which is safe in the browser because it is a public `pk_` key restricted by allowed website addresses.
- A signed-in customer's bearer token is attached automatically, so submissions land on their own record.
- On `401`, the response interceptor retries once without the token using plain `axios` (so the request interceptor cannot put it back). If the answer changes, the session was the problem; if it does not, the customer is logged out.
- `errorMessage()` prefers the API's own human-readable message, but hides key/server errors behind a fallback.
- `newRequestKey()` and `once()` implement `Idempotency-Key` so a double submit returns the first answer instead of creating a second record.
- `noteTestMode()` shows an info toast when a `pk_test_` key checked the request but saved nothing.

`src/shared/lib/payments.ts` — paying an invoice (72 lines)

Purpose: Starts a payment, opens Paystack in a popup, verifies it, and falls back to the invoice's own payment page.

```
export const payInvoice = async ({
  invoiceId,
  email,
  paymentLink,
  allowRedirect = true,
}: PayInvoiceOptions): Promise<PaymentOutcome> => {
  let start: PaymentStart;
  try {
    const { data } = await realEstateApi.post<Sent<PaymentStart>>("/payments/start", { invoiceId, customerEmail: ema
il });
    start = data.data;
  } catch (error) {
    if (allowRedirect && paymentLink) return goTo(paymentLink);
    throw error;
  }

  const accessCode = start.accessCode;
  if (start.provider !== "paystack" || !accessCode) {
    const link = start.authorizationUrl ?? paymentLink;
    if (allowRedirect && link) return goTo(link);
    throw new Error("Online payment isn't available right now.");
  }

  return new Promise<PaymentOutcome>((resolve) => {
    new PaystackPop().resumeTransaction(accessCode, {
      onSuccess: async (transaction: { reference: string }) => {
        try {
          const { data } = await realEstateApi.post<Sent<{ paid: boolean }>>("/payments/verify", {
```

```
            invoiceId,
            reference: transaction.reference,
          });
          resolve(data.data.paid ? "paid" : "pending");
        } catch {
          resolve("pending");
        }
      },
      onCancel: () => resolve("cancelled"),
      onError: () => resolve("failed"),
    });
  });
};
```

How it works

- `PaymentOutcome` is `paid | pending | cancelled | failed | redirected`.
- The amount always comes from SabiFlow, never from the browser.
- If `/payments/start` fails, the invoice's `paymentLink` is used when `allowRedirect` is true.
- If Paystack returns no access code, the authorization URL or payment link is used instead.
- On success, `/payments/verify` confirms the payment. If verification fails, the outcome is `pending` because Paystack also notifies SabiFlow directly.
- `allowRedirect: false` keeps payment inside the app (used by the account page); otherwise the browser navigates to the invoice page.

`src/shared/lib/utils.ts` — the shared toolbox (64 lines)

Purpose: Class-name merging, compact currency formatting, URL upgrades, HTML stripping, and cookie helpers.

```
export const formatCurrency = (amount: number) => {
  if (amount >= 1000000000) {
    return `${(amount / 1000000000).toFixed(3).replace(/\.?0+$/, "")}B`;
  }
  if (amount >= 10000000) {
    return `${(amount / 10000000).toFixed(3).replace(/\.?0+$/, "")}M`;
  }
  return amount.toLocaleString();
};
```

How it works

- `cn()` = `twMerge(clsx(...))`; `clsx` handles conditional class names and `tailwind-merge` resolves conflicts (the later class wins).
- `formatCurrency` abbreviates billions and tens-of-millions, then falls back to `toLocaleString()`.
- `ensureHttps` upgrades `http://` CDN images to `https://` to avoid mixed-content blocking.
- `stripHtmlTags` converts API rich text into clean plain text for copy/share.
- Cookies use `path=/`, `SameSite=Lax`, and `Secure` automatically on HTTPS; the default lifetime is 365 days.

`src/shared/store/useThemeStore.ts` — light/dark mode (25 lines)

```
export const useThemeStore = create<ThemeState>((set) => ({
  theme: (localStorage.getItem("theme") as Theme) || "light",

  toggleTheme: () =>
    set((state) => {
      const newTheme = state.theme === "light" ? "dark" : "light";
      localStorage.setItem("theme", newTheme);
      return { theme: newTheme };
    }),

  setTheme: (theme) => {
    localStorage.setItem("theme", theme);
    set({ theme });
  },
```

```
}));
```

How it works

- The store reads `localStorage` at module load, so there is no flash of the wrong theme.
- It does not touch the DOM. `App.tsx` watches `theme` and toggles the `dark` class on `<html>`.
- `localStorage` is accessed directly (no SSR guard).

`src/shared/hooks/useScrollToHash.ts` — deep-link scrolling (23 lines)

Purpose: Scrolls to the element named in the URL hash whenever the location changes.

```
const useScrollToHash = (): void => {
  const location = useLocation();

  useEffect(() => {
    if (!location.hash) return;
    const id = location.hash.replace("#", "");
    const element = document.getElementById(id) as HTMLElement | null;
    if (element) {
      element.scrollIntoView({ behavior: "smooth", block: "start" });
    }
  }, [location]);
};
```

Used by `Home`, `About`, `Service`, `Contact`, and `PropertiesPage`, so footer links such as `/#features` or `/About#Journey` land on the right section.

`src/shared/hooks/useRequestKey.ts` — idempotency per submission (20 lines)

Purpose: Gives each form one stable idempotency key, renewable after a successful submission.

```
const useRequestKey = () => {
  const key = useRef(newRequestKey());
  return {
    current: () => key.current,
    /** Call once it has gone through, so the next one is new. */
    renew: () => {
      key.current = newRequestKey();
    },
  };
};
```

How it works

- Pressing submit twice, or again after a timeout, returns the first answer instead of making a second record.
- A request that failed can be sent again with the same key.
- The key is per hook instance; callers decide when to `renew()`.

`src/shared/components/ui/LazyImage.tsx` — lazy loading with fallback (113 lines)

Purpose: Renders an image only when its container approaches the viewport, with a pulsing placeholder and an error fallback.

```
{!isLoaded && (
  <div
    className={`absolute inset-0 bg-gray-200 dark:bg-neutral-800 animate-pulse z-0 ${placeholderClassName}`}
  />
)}

{isInView && (
  <img
    src={imageSource}
    alt={alt}
    loading="lazy"
```

```
    decoding="async"
    onLoad={handleImageLoad}
    onError={handleImageError}
    className={`transition-opacity duration-300 ease-in-out ${
      isLoaded ? "opacity-100" : "opacity-0"
    } ${className}`}
    {...props}
  />
)}
```

How it works

- The `<img>` is not rendered at all until the container is within `rootMargin` (default `250px`) of the viewport, so the browser never requests off-screen images.
- A pulsing grey block stands in until the image decodes.
- On load, opacity animates from `0` to `1` over 300ms (no layout shift).
- On error, the source swaps to `fallbackSrc` (`/logo/Image.png` by default).
- The load/error state resets whenever `src` changes.

`src/shared/components/ui/Skeleton.tsx` + `Skeletons.tsx` — loading placeholders (15 + 371 lines)

```
function Skeleton({ className, ...props }: React.HTMLAttributes<HTMLDivElement>) {
  return (
    <div
      className={cn("animate-pulse rounded-md bg-black/10 dark:bg-white/10", className)}
      {...props}
    />
  );
}
```

`Skeletons.tsx` composes that primitive into layout-matching placeholders:

Component	Mirrors
`PropertyCardSkeleton`	a `PropertyCard` (image, title, location, description, pills, price, button)
`CommentCardSkeleton`	a testimonial card
`FAQCardSkeleton`	an FAQ card
`PropertyDetailsSkeleton`	the entire detail page: header, gallery, video, two-column body, related grid, bottom action bar
`ProfileDropdownSkeleton`	the profile dropdown
`ProfilePageSkeleton`	the profile page two-column layout

The skeleton markup mirrors the real DOM structure, so the layout does not jump when data arrives.

---

4. Auth: store, services, and pages

`src/features/auth/store/useAuthStore.ts` — the auth brain (122 lines)

Purpose: Zustand store holding the token, customer profile, authentication flag, and company ID. It persists to `localStorage` and mirrors the session into cookies.

```
export const useAuthStore = create<AuthStore>()(
  persist(
    (set) => ({
      token: initialToken,
      customer: initialCustomer,
      isAuthenticated: !!initialToken || !!initialCustomer,
      companyId: COMPANY_ID,
      setToken: (token) => {
        if (token) {
          setCookie(AUTH_TOKEN_COOKIE, token);
        } else {
          deleteCookie(AUTH_TOKEN_COOKIE);
        }
        set({ token, isAuthenticated: !!token });
      },
      setCustomer: (customer) => {
        if (customer) {
          setCookie(AUTH_CUSTOMER_COOKIE, JSON.stringify(customer));
        } else {
          deleteCookie(AUTH_CUSTOMER_COOKIE);
        }
        set({ customer, isAuthenticated: !!customer });
      },
      setCompanyId: (companyId) => set({ companyId }),
      logout: () => {
        deleteCookie(AUTH_TOKEN_COOKIE);
        deleteCookie(AUTH_CUSTOMER_COOKIE);
        set({ token: null, customer: null, isAuthenticated: false });
      },
    }),
    {
      name: "auth-storage",
      // ...
    }
  )
);
```

How it works — the dual persistence model

- Auth state is written to two places: Zustand's `persist` middleware (`localStorage["auth-storage"]`) and cookies (`rewaciti_auth_token`, `rewaciti_auth_customer`).
- Cookies win. `merge()` prefers cookie values over persisted ones, and `onRehydrateStorage` re-applies them after `localStorage` is read.
- Cookies survive a `localStorage` clear and can be read server-side later.
- `setToken` / `setCustomer` always keep the cookie and the store in sync.
- `logout()` deletes both cookies and clears the state.
- `COMPANY_ID` comes from `VITE_COMPANY_ID` and is attached to auth and CRM requests.
- Cookie reads are guarded by `typeof document`, and malformed customer JSON is ignored.

`src/features/auth/services/authAPI.ts` — every customer endpoint (280 lines)

Purpose: Axios calls for customer sign-up, sign-in, Google sign-in, verification, password reset, and profile management.

```
export const authAPI = {
  login: async (payload: LoginPayload): Promise<LoginResponse> => {
    const response = await axios.post(`${API_URL}/customers/login`, payload);
    return response.data;
  },
```

```
  signUp: async (payload: SignUpPayload): Promise<SignUpResponse> => {
    const response = await axios.post(`${API_URL}/customers/register`, payload);
    return response.data;
  },

  // ...
  getProfile: async (): Promise<ProfileResponse> => {
    const token = useAuthStore.getState().token;
    const response = await axios.get(`${API_URL}/customers/profile`, {
      headers: { Authorization: `Bearer ${token}` },
    });
    return response.data;
  },
};
```

Endpoint map

Method	Endpoint	Auth
POST	`/customers/login`	—
POST	`/customers/google/sign-in`	—
POST	`/customers/register`	—
POST	`/customers/verify-email`	—
POST	`/customers/resend-verification-otp`	—
POST	`/customers/forgot-password`	—
POST	`/customers/reset-password`	—
GET	`/customers/profile`	Bearer
PUT	`/customers/profile`	Bearer
PUT	`/customers/change-password`	Bearer

How it works

- There is no interceptor here. Each authenticated method reads `useAuthStore.getState().token` and sets the `Authorization` header itself.
- Every payload includes `companyId` (multi-tenant).
- The token is read at call time, so calls always use the current session.
- Errors are not caught here; callers receive raw Axios rejections and format them locally.
- The file also exports the request/response interfaces (`LoginPayload`, `ProfileResponse`, `UpdateProfilePayload`, and so on).

`src/features/auth/services/accountAPI.ts` — the customer's own records (74 lines)

Purpose: Reads and mutates the signed-in customer's visits, inquiries, requests, and deals through `realEstateApi`.

```
const list = async <T,>(path: string) =>
  (await realEstateApi.get<Paged<T>>(path, { params: { limit: 50 } })).data.data;

export const accountAPI = {
  visits: () => list<AccountVisit>("/me/bookings"),
  rescheduleVisit: async (id: string, body: RescheduleVisitRequest) =>
    (await realEstateApi.post<Sent<AccountVisit>>(`/me/bookings/${encodeURIComponent(id)}/reschedule`, body)).data,
  cancelVisit: async (id: string, body: CancelVisitRequest) =>
    (await realEstateApi.post<Sent<AccountVisit>>(`/me/bookings/${encodeURIComponent(id)}/cancel`, body)).data,
  inquiries: () => list<AccountInquiry>("/me/inquiries"),
  requests: () => list<AccountRequest>("/me/requests"),
  deals: () => list<AccountDeal>("/me/deals"),
};
```

How it works

- All list helpers request `limit: 50`; no pagination parameters are exposed.

- Reschedule and cancel URL-encode the booking ID and return the updated visit.
- Auth headers are handled by `realEstateApi`.
- Endpoints: `GET /me/bookings`, `POST /me/bookings/{id}/reschedule`, `POST /me/bookings/{id}/cancel`, `GET /me/inquiries`, `GET /me/requests`, `GET /me/deals`.

`src/features/auth/services/googleSignIn.ts` — Google sign-in with PKCE (88 lines)

Purpose: Starts Google sign-in and consumes the result without exposing a client secret.

```
export async function startGoogleSignIn(companyId: string, next = "/") {
  const codeVerifier = base64url(crypto.getRandomValues(new Uint8Array(32)));
  // S256 needs crypto.subtle (https and localhost). Elsewhere, such as a phone
  // trying the site over the local network, the verifier itself is sent.
  let codeChallenge = codeVerifier;
  let codeChallengeMethod = "plain";
  if (crypto.subtle) {
    const digest = await crypto.subtle.digest("SHA-256", new TextEncoder().encode(codeVerifier));
    codeChallenge = base64url(new Uint8Array(digest));
    codeChallengeMethod = "S256";
  }

  sessionStorage.setItem(PENDING_KEY, JSON.stringify({ codeVerifier, next: pageOnThisSite(next) }));

  const params = new URLSearchParams({
    companyId,
    redirectUrl: `${window.location.origin}${GOOGLE_CALLBACK_PATH}`,
    codeChallenge,
    codeChallengeMethod,
  });
  window.location.assign(`${API_URL}/customers/google/start?${params.toString()}`);
}
```

How it works

- The verifier and the post-login `next` page are stored in `sessionStorage` under a pending key.
- `next` is sanitized so it can only be a same-site path (starts with `/`, not `//`).
- `canUseGoogleSignIn()` returns false inside in-app browsers (Facebook, Instagram, TikTok, Snapchat, and similar), where the popup/redirect flow is unreliable.
- `takeGoogleResult()` reads `google_handoff`, `google_error`, and `google_cancelled` from the URL hash, then strips the hash with `history.replaceState`.
- `takePendingGoogleSignIn()` is single-use: it removes the pending entry before parsing.
- The callback path is `/auth/google` and must be registered exactly in SabiFlow's Google sign-in settings.

`src/features/auth/pages/Login.tsx` — sign-in and sign-up in one page (473 lines)

Purpose: A single screen with two modes (`isSignUp`), email/password sign-in, optional Google sign-in, a remembered email, and the sign-up OTP handoff.

Key state

- `isSignUp`, `showPassword`, `showConfirmPassword`, `agree`, `isLoading`, `isGoogleLoading`, `showGoogle`.
- `email` initialised from `localStorage["rewaciti_remembered_email"]`, plus `rememberEmail` and `password`.
- Sign-up only: `firstName`, `lastName`, `phoneNumber`, `confirmPassword`.
- A `pageshow` effect unlocks the form when the page is restored from the back/forward cache.

```
const response = await authAPI.login({
  companyId,
  email,
  password,
});

if (response.token) {
  setToken(response.token);
  setCustomer(response.customer);
```

```
  setCompanyId(response.customer.companyId);
  await usePropertyStore.getState().fetchWishlist();
  toast.success("Login successful!");
  navigate("/");
}
```

How it works

- Sign-in validity is only `email && password`. Sign-up also requires every field, matching passwords, and the agreement checkbox.
- `handleLogin` stores or removes the remembered email, calls `authAPI.login`, updates the auth store, refreshes the wishlist badge, and navigates to `/`.
- `handleSignUp` calls `authAPI.signUp`, optionally saves the returned `companyId`, writes `{ email, firstName, lastName }` to `sessionStorage["pendingVerification"]`, and navigates to `/auth/verify-email`.
- The sign-up flow does not log the user in; the OTP page completes it.
- Google sign-in is shown only when `canUseGoogleSignIn()` is true and calls `startGoogleSignIn(companyId)`.
- Toggling modes clears the form fields but does not restore the remembered email.
- A local `getErrorMessage` helper prefers `response.data.message`, then `message`, then a fallback.

Gotchas

- No email-format or password-strength validation.
- There is no "already signed in" redirect; success always goes to `/`.
- Only the email is remembered, not the session.

`src/features/auth/pages/VerifyEmail.tsx` — 6-box OTP (317 lines)

Purpose: Verifies the six-digit code sent after sign-up, with a countdown and resend.

```
useEffect(() => {
  // Get pending verification data
  const data = sessionStorage.getItem("pendingVerification");
  if (!data) {
    navigate("/auth/login");
    return;
  }
  setPendingData(JSON.parse(data));
}, [navigate]);

// Timer for OTP expiry
useEffect(() => {
  if (timeLeft <= 0) {
    setCanResend(true);
    return;
  }

  const timer = setInterval(() => {
    setTimeLeft((prev) => prev - 1);
  }, 1000);

  return () => clearInterval(timer);
}, [timeLeft]);
```

How it works

- Missing `pendingVerification` redirects to `/auth/login`.
- The six boxes are independent inputs. The component moves focus by DOM id (`otp-0` … `otp-5`), auto-advancing on entry and retreating on Backspace.
- Verification requires all six digits and the stored email, then calls `authAPI.verifyEmail({ companyId, email, otp })`.
- On `response.token && response.customer`, it updates the auth store, removes `pendingVerification`, toasts, and navigates to `/`.
- Resend calls `authAPI.resendOTP`, clears the OTP, resets the timer to 300 seconds, and hides the resend hint.

Gotchas

- The resend button is disabled only by `isLoading`; the countdown is informational, so resend is possible immediately.
- The initial countdown is 15 minutes; resend resets it to 5 minutes.
- OTP inputs are not disabled while verification is in progress.
- The stored pending object excludes the password; `companyId` comes from the store.

`src/features/auth/pages/ForgotPassword.tsx` — two-step wizard (335 lines)

Purpose: Collects an email, sends a reset code, then collects a six-digit OTP and hands it to the reset page. It does not verify the OTP itself.

```
const handleVerifyOtp = () => {
  if (!isOtpComplete) {
    setFormError("Please enter all 6 digits of the code.");
    return;
  }

  setFormError("");
  sessionStorage.setItem("resetEmail", email);
  sessionStorage.setItem("resetOtp", otp.join(""));
  navigate("/auth/reset-password");
};
```

How it works

- Step 1 calls `authAPI.forgotPassword`, stores the email in `sessionStorage["resetEmail"]`, marks `hasSentCode`, and switches to the OTP step.
- Resend calls the same endpoint and clears the OTP.
- Step 2 only checks that six digits were entered. The OTP is stored in `sessionStorage["resetOtp"]` and verified later by the reset endpoint.
- Editing the email resets the sent state and clears the OTP.
- "Back to Email" returns to step 1 while retaining the email.
- OTP boxes auto-advance and support Backspace navigation.

Gotchas

- The OTP is not validated against the server on this page.
- There is no expiry timer here.
- `resetEmail` persists in `sessionStorage` even if the user backs out.

`src/features/auth/pages/ResetPassword.tsx` — final reset step (213 lines)

Purpose: Reads the email and OTP saved by the forgot-password page, collects a new password twice, and submits the reset.

```
useEffect(() => {
  const resetEmail = sessionStorage.getItem("resetEmail");
  const otpFromStorage = sessionStorage.getItem("resetOtp");
  if (!resetEmail || !otpFromStorage) {
    navigate("/auth/forgot-password");
    return;
  }
  setEmail(resetEmail);
  setResetOtp(otpFromStorage);
}, [navigate]);

const isPasswordValid =
  newPassword &&
  confirmPassword &&
  newPassword === confirmPassword &&
  newPassword.length >= 8;
```

How it works

- If either stored value is missing, the page redirects back to `/auth/forgot-password`.

- The stored email is displayed; the OTP is not asked for again.
- Validation checks the combined rule, then mismatch, then minimum length with separate messages.
- Calls `authAPI.resetPassword({ companyId, email, otp: resetOtp, newPassword })`.
- On success it toasts, removes `resetEmail`, and navigates to `/auth/login`.
- Both password fields have independent show/hide toggles.

Gotchas

- `resetOtp` is never removed from `sessionStorage` after success.
- The only password rule is "at least 8 characters".
- The page renders briefly before redirecting when storage is missing.

`src/features/auth/pages/GoogleCallback.tsx` — completes Google sign-in (85 lines)

Purpose: Landing page at `/auth/google` that exchanges the handoff code for a session, refreshes the wishlist, and redirects.

```
useEffect(() => {
  if (started.current) return;
  started.current = true;

  const result = takeGoogleResult();
  const pending = takePendingGoogleSignIn();
  const backToSignIn = (message?: string) => {
    if (message) toast.error(message);
    navigate("/auth/login", { replace: true });
  };

  // They chose Cancel on Google's page: nothing to tell them
  if (result.cancelled) return backToSignIn();
  if (result.error) return backToSignIn(result.error);
  // No verifier: this browser didn't start this sign-in (a link someone sent, say)
  if (!result.handoff || !pending) return backToSignIn("That Google sign-in didn’t work. Please try again.");

  const handoff = result.handoff;
  // ...
});
```

How it works

- A `started` ref guards against React StrictMode running the effect twice, because the result is single-use.
- Cancellation silently returns to login; errors toast and redirect.
- A missing handoff or pending verifier means this browser did not start the sign-in.
- On success it calls `authAPI.googleSignIn({ companyId, handoff, codeVerifier })`, then updates the auth store, awaits `fetchWishlist()`, and navigates to `pending.next` with `replace`.
- The toast says "Welcome to RewaCiti!" when `response.created` is true, otherwise "Login successful!".
- While processing, it renders a spinner page with `Navbar`, a hero image, and a `noindex` meta tag.

`src/features/auth/components/ProfileDropdown.tsx` — account menu (76 lines)

Purpose: Dropdown showing the signed-in customer's initials, name, and email, with profile and logout actions.

```
function ProfileDropdown({ isOpen, onClose }: ProfileDropdownProps) {
  const { customer, logout } = useAuthStore();
  const navigate = useNavigate();

  const firstName = customer?.firstName ?? "";
  const lastName = customer?.lastName ?? "";
  const email = customer?.email ?? "";

  const initials =
    `${firstName.charAt(0)}${lastName.charAt(0)}`.toUpperCase() || "U";

  const handleLogout = () => {
    logout();
```

```
    onClose();
    navigate("/");
  };
```

How it works

- Visibility is CSS-driven: `isOpen` toggles opacity, scale, translate, and pointer events, so the element stays mounted and the open/close transitions animate.
- "View Profile" links to `/auth/profile` and calls `onClose`.
- Logout clears the store and navigates to `/`.
- The `isMobile` prop is declared but not used.

`src/features/auth/pages/Profile.tsx` — profile settings (747 lines)

Purpose: Profile editor with "Profile" and "Security" tabs. It supports personal/address edits, a base64 profile photo, password changes, and logout.

```
const handleChangePassword = async (e: React.FormEvent) => {
  e.preventDefault();
  setFormError("");

  if (!currentPassword || !newPassword) {
    setFormError("Please fill in all password fields.");
    return;
  }
  if (newPassword.length < 8) {
    setFormError("Password must be at least 8 characters long.");
    return;
  }
  if (newPassword !== confirmPassword) {
    setFormError("New passwords do not match.");
    return;
  }
  if (currentPassword === newPassword) {
    setFormError("New password must be different from your current password.");
    return;
  }
  // ...
};
```

How it works

- On mount, `authAPI.getProfile()` fills the form. `loadFromProfile` prefers the default address (`isDefault`), then the first address, then the flat fields.
- Only one section can be edited at a time (`editingSection`), and `cancelEdit()` restores the last server snapshot.
- Saving a section sends the full personal + address payload to `authAPI.updateProfile`, then reloads the returned customer and syncs the auth store.
- Password change validates presence, length, confirmation, and difference from the current password before calling `authAPI.changePassword`.
- Photo upload rejects non-images and files over 5MB, previews instantly with `FileReader`, then submits the base64 data URL through `updateProfile`.
- Date helpers convert between the API's `DD-MM-YYYY` and the date input's `YYYY-MM-DD`.
- Logout clears the store, toasts, and navigates to `/`.

Gotchas

- Saving the address section also sends `firstName`/`lastName`, even if empty.
- The profile picture is stored as a base64 data URL in the JSON body; there is no file-upload endpoint.
- There is no auth guard; a failed profile fetch leaves the page rendered after a toast.

`src/features/auth/pages/AccountPage.tsx` — the account dashboard (1454 lines)

Purpose: Full account dashboard with Saved, Visits, Payments, Messages, and Profile tabs. It loads tab-specific data and manages visits, payments, receipts, and basic profile edits.

```
const loadAccountSection = useCallback(async () => {
  const requestId = ++loadRequestId.current;
  setIsLoading(true);
  try {
    switch (activeTab) {
      case "saved": {
        const saved = await propertiesAPI.saved();
        if (requestId !== loadRequestId.current) return;
        usePropertyStore.setState({
          shortlistedProperties: saved.map((listing) => toProperty(listing)),
        });
        break;
      }
      case "visits": {
        const result = await accountAPI.visits();
        if (requestId !== loadRequestId.current) return;
        setVisits(result);
        break;
      }
      // ...
    }
  } finally {
    // ...
  }
}, [activeTab]);
```

How it works

- `activeTab` initialises from the `?tab=` query parameter (via `isAccountTab`), otherwise `"profile"`.
- Tab clicks update the query string: `profile` clears the params, the others set `?tab=`.
- Loading is tab-specific:
- `saved` → `propertiesAPI.saved()` and update the property store.
- `visits` → `accountAPI.visits()`.
- `payments` → `accountAPI.deals()`.
- `messages` → `Promise.all([accountAPI.requests(), accountAPI.inquiries()])`.
- `profile` → `authAPI.getProfile()`.
- `loadRequestId` guards against stale responses: a reply is ignored if the tab changed while it was in flight.
- Visits default to "Coming up": today or later, excluding `cancelled`, `completed`, `no_show`, and `expired`. "All visits" shows everything.
- Messages merge inquiries and requests, sorted newest-first by `createdAt`.
- Saved properties can be removed with `propertiesAPI.unsave`, which refreshes the store.
- Profile details save only first name, last name, and phone; email is read-only.
- Password change only requires a new password of at least 8 characters.
- Payments call `payInvoice` with `allowRedirect: false`; receipts use `downloadFile`, which fetches same-origin blobs but opens external origins in a new tab.

Gotchas

- No auth guard; the page relies on the API and store and does not redirect unauthenticated visitors.
- Payments need an email from `profile?.email || customer?.email`; otherwise the user is told to add one.
- Visit actions depend on server-provided `canReschedule` / `canCancel` flags or fallback rules.
- Responses flagged as test mode are ignored by `finishVisitAction`.

---

5. Properties: services and the store

The property feature is split into a service layer (HTTP) and a Zustand store (state). Components never call Axios directly.

`src/features/properties/services/propertiesAPI.ts` — listing endpoints (59 lines)

Purpose: Wraps every public listing endpoint: search, detail, related, filters, locations, types, forms, saved listings, and purchase holds.

```
export const propertiesAPI = {
  // ...
  saved: async (): Promise<PublicListing[]> =>
    (await realEstateApi.get<{ data: PublicListing[] }>("/me/saved-properties")).data.data,

  save: async (id: string): Promise<PublicListing[]> =>
    (await realEstateApi.put<Sent<PublicListing[]>>(`/me/saved-properties/${id}`)).data.data,

  unsave: async (id: string): Promise<PublicListing[]> =>
    (await realEstateApi.delete<Sent<PublicListing[]>>(`/me/saved-properties/${id}`)).data.data,

  /** Holds a place while the customer pays. SabiFlow works out the amount. */
  purchase: async (idOrSlug: string, body: Contact & { plan?: "full" | "deposit" }, key: string): Promise<PurchaseRe
sult> =>
    (await realEstateApi.post<PurchaseResult>(`${path(idOrSlug)}/purchase`, body, once(key))).data,
};
```

Endpoints

Method	Endpoint	Returns
GET	`/properties`	`Paged<PublicListing>`
GET	`/properties/{idOrSlug}`	`PublicListingDetail`
GET	`/properties/{idOrSlug}/related?sameAgent`	`PublicListing[]`
GET	`/properties/filters`	`PropertyFilters`
GET	`/locations`	`PropertyLocations["data"]`
GET	`/property-types`	`PropertyType[]`
GET	`/forms`	`PublicForms`
GET	`/me/saved-properties`	`PublicListing[]`
PUT	`/me/saved-properties/{id}`	`PublicListing[]`
DELETE	`/me/saved-properties/{id}`	`PublicListing[]`
POST	`/properties/{idOrSlug}/purchase`	`PurchaseResult`

How it works

- `path()` URL-encodes the id or slug.
- List endpoints unwrap `.data`; endpoints that wrap an array use `.data.data`.
- Save uses `PUT`, unsave uses `DELETE`, and both return the updated saved list.
- `purchase` sends `Contact & { plan? }` with an `Idempotency-Key` via `once(key)`.
- `purchase` only holds the property; the amount and payment schedule are decided by SabiFlow.

`src/features/properties/services/requestsAPI.ts` — every public form (60 lines)

Purpose: Sends inquiries, contact messages, property requests, co-rent requests, agent applications/reports, and ratings.

```
const send = async <T,>(url: string, body: object, key: string) =>
  (await realEstateApi.post<Sent<T>>(url, body, once(key))).data;
```

```
export const requestsAPI = {
  /** A question about one property. */
  inquiry: (body: Contact & { propertyId: string; message?: string }, key: string) => send("/inquiries", body, key),

  /** A general message. */
  contact: (body: Contact & { message: string; topic?: string; heardFrom?: string }, key: string) => send("/contact"
, body, key),

  /** What they're looking for. They're emailed when a listing fits. */
  propertyRequest: (body: PropertyRequest, key: string) => send("/property-requests", body, key),

  /** Someone to share a place with. */
  coRent: (body: CoRentRequest, key: string) => send("/co-rent-requests", body, key),

  // ...
};
```

Endpoints

- `POST /inquiries`
- `POST /contact`
- `POST /property-requests`
- `POST /co-rent-requests`
- `POST /agent-applications`
- `POST /agent-reports`
- `POST /ratings`

How it works

- `send<T>` posts a `Sent<T>` with `once(key)` for idempotency; callers supply the key from `useRequestKey`.
- `rating` is the exception: it has no idempotency key and returns `Sent<{ id: string }>`.
- Guests prove ownership of a rating with an optional email; `type` must be `"visit"` or `"deal"`.
- Sharing values are narrowed unions (`whole` / `shared_room` / `shared_apartment`).

`src/features/properties/services/toProperty.ts` — API shape → app shape (118 lines)

Purpose: The single translator between SabiFlow's `PublicListing` and the `Property` the UI renders. It also formats prices, durations, specifications, notes, and addresses.

```
export const toProperty = (listing: PublicListing | PublicListingDetail, fields: FormField[] = []): Property => {
  const detail = "visitFee" in listing ? listing : null;
  const images = listing.photos.map(ensureHttps);

  return {
    id: listing.id,
    slug: listing.slug,
    reference: listing.reference ?? undefined,
    name: listing.title,
    img: images[0] ?? "",
    images,
    description: detail?.description ?? "",
    bedrooms: listing.bedrooms ?? 0,
    bathrooms: listing.bathrooms ?? 0,
    category: listing.type ?? "Property",
    purpose: listing.purpose,
    listingType: PURPOSE_LABELS[listing.purpose] ?? "",
    duration: durationOf(listing),
    sharing: listing.sharing,
    places: listing.places,
    reserved: listing.status === "reserved",
    pricing: listing.pricing,
    agentId: listing.agentId,
    agent: detail?.agent,
    // ...
  };
};
```

How it works

- A detail listing is detected with `"visitFee" in listing`; only detail listings yield `description`, `agent`, and `visitationfee`.
- Photos and `videoUrl` pass through `ensureHttps`; the first photo becomes `img` (or `""`).
- `durationOf` prefers `listing.term` (pluralising by removing a trailing `s` when the value is 1), otherwise maps `pricePeriod`.
- `specificationsOf` adds size (`m²`), sharing (with people per room), places left when total > 1, then labelled custom fields except `special_notes`.
- `textOf` joins arrays, converts booleans to Yes/No, and formats money as `₦` with `toLocaleString`.
- `specialNotes` comes from `customFields.special_notes` as a filtered string list.
- `addressOf(property)` formats `"area, city, state state."` and is used by cards, sharing, and modals.

`src/features/inspections/services/visitsAPI.ts` — visit slots and booking (25 lines)

Purpose: Fetches available inspection slots and creates bookings.

```
export const visitsAPI = {
  /** Visit times on a day, in RewaCiti's time zone. Notice, hours and taken times are already applied. */
  slots: async (propertyId: string, date: string): Promise<VisitSlot[]> =>
    (await realEstateApi.get<{ slots: VisitSlot[] }>(`/properties/${propertyId}/slots`, { params: { date } })).data.
slots,

  /** A visit with a fee is held until it's paid at `paymentLink` or with `invoiceId`. */
  book: async (body: BookingRequest, key: string): Promise<Sent<Booking>> =>
    (await realEstateApi.post<Sent<Booking>>("/bookings", body, once(key))).data,
};
```

How it works

- Slots are already timezone-adjusted by the backend; notice periods, opening hours, and taken times are applied.
- `start` must match one of the returned slot times.
- `alsoSee` lets several listings be viewed in one visit for one fee.
- A booking with a fee is held until paid at `paymentLink` or through `invoiceId`.

`src/features/properties/store/usePropertyStore.ts` — the core store (272 lines)

Purpose: Holds listings, featured listings, categories, forms, locations, related listings, the current property, pagination, and the wishlist. Almost every interesting property behaviour lives here.

How it works

- **Caches for the visit:** `formsRequest`, `locationsRequest`, and `relatedCache` hold promises so several renders share one request. `listRequest` is a counter used to discard out-of-order search replies.
- `fetchProperties(apiPage, filters)`**:** strips blank filters, calls `propertiesAPI.list`, maps through `toProperty`, and only applies the reply if it is still the newest request.
- `fetchFeaturedProperties()`**:** returns early if already loaded; otherwise fetches page 1 with `limit: 8`.
- `fetchCategories()` / `fetchForms()` / `fetchLocations()`**:** cache-once helpers; failures reset the cached promise so a retry is possible.
- `fetchRelatedProperties(property, sameAgentOnly, page, limit)`**:** caches the full related list by `${id}:${sameAgentOnly}`, then slices it for the requested page.
- `fetchPropertyBySlug(slug)`**:** cache-first. It serves the current property, list, related list, or featured list immediately, then revalidates from the API in the background and upserts the result into `properties`.
- **Pagination:** `nextPage` / `prevPage` clamp to `1 … ceil(total / ITEMS_PER_PAGE)`; `setPage` only updates the UI counters.
- **Wishlist:** `fetchWishlist()` clears when signed out; `toggleShortlist()` updates optimistically, then reconciles with the server's returned saved list and rolls back on error.

```
fetchPropertyBySlug: async (slug: string) => {
  // Serve from cache first so back-navigation is instant,
  // but still revalidate in the background for fresh data.
  const { properties, relatedProperties, featuredProperties, currentProperty } = get();
```

```
  const cached =
    (currentProperty?.slug === slug ? currentProperty : null) ??
    properties.find((p) => p.slug === slug) ??
    relatedProperties.find((p) => p.slug === slug) ??
    featuredProperties.find((p) => p.slug === slug) ??
    null;

  if (cached) {
    set({ currentProperty: cached });
  } else {
    set({ detailsLoading: true, currentProperty: null });
  }

  try {
    const [listing, forms] = await Promise.all([propertiesAPI.get(slug), get().fetchForms()]);
    const mapped = toProperty(listing, forms?.property);
    // ...
  } catch (err) {
    // ...
  }
},
```

Gotchas

- The inventory API base URL is not hard-coded here; it comes from the shared `realEstateApi`.
- Only the newest search may fill the list, so fast typing cannot leave stale results.
- `fetchPropertyBySlug` keeps a cached copy if the revalidation fails, so the page does not flash "Not Found".

---

6. Properties: the listing page

`src/features/properties/pages/PropertiesPage.tsx` — search, filters, list/map (902 lines)

Purpose: The main browse page: a sticky search bar, cascading location filters, a full filter sheet, list/map switching, pagination, and service call-to-actions.

```
useEffect(() => {
  fetchProperties(1, {
    search: searchTerm || undefined,
    state: state || undefined,
    city: city || undefined,
    area: area || undefined,
    nearby: university || undefined,
    typeId: category || undefined,
    ...(bedrooms === "shared"
      ? { sharing: "shared_room" }
      : { bedrooms: bedrooms ? Number(bedrooms) : undefined }),
    minPrice: priceRange[0] > 0 ? priceRange[0] : undefined,
    maxPrice: priceRange[1] < 999999999 ? priceRange[1] : undefined,
    purpose: listingType || undefined,
  });
}, [/* filter state */]);
```

How it works

- `flattenLocations` turns the location hierarchy into rows per area, per nearby landmark/university, or a fallback state/city row.
- State, city, area, and university options are derived with `useMemo`; each level is disabled until its parent is selected.
- Cascading handlers clear invalid child selections when a parent changes. Selecting an area can fill its city; selecting a university can fill its city.
- Every search/filter change refetches page 1. Shared-room-only is sent as `sharing: "shared_room"`.
- Pagination is `Math.max(1, Math.ceil(totalProperties / ITEMS_PER_PAGE))`, and `nextPage`/`prevPage` are guarded by `apiPage`.
- List mode renders skeletons while loading, a retry panel on error, a "No properties found" panel with a request link, or property cards.
- Map mode renders a loading panel, an empty state, or a two-column layout: a scrollable card list on large screens and `PropertyMap`.
- Hovering a card highlights its map marker; selecting a map item scrolls its card into view.
- `activeFilterCount` is derived from populated filters; `clearAllFilters` resets everything, including `priceRange`.
- Three service cards below the results link to `/request-property`, `/co-rent`, and `/list-property`.
- The page measures the navbar and sticky bar heights so the map offset is correct.

Gotchas

- The search effect has no debounce, so typing can trigger a request per keystroke.
- Only `state` and `area` are initialised from URL search params; other filters are not.
- In map mode the card list is hidden on small screens (`hidden ... lg:block`), leaving only the map.

---

7. Properties: the detail page

`src/features/properties/pages/PropertyDetailsPage.tsx` — one listing in full (1279 lines)

Purpose: Displays a single property by slug: gallery/video, specifications, amenities, rules/bills modals, map, booking/payment/inquiry actions, and related listings.

```
const cachedProperty =
  properties.find((p) => p.slug === slug) ??
  relatedProperties.find((p) => p.slug === slug) ??
  featuredProperties.find((p) => p.slug === slug) ??
  null;
const property =
  currentProperty && currentProperty.slug === slug
    ? currentProperty
    : cachedProperty ?? currentProperty;
const descriptionText = (property?.description ?? "")
  .replace(/<[^>]+>/g, " ")
  .replace(/\s+/g, " ")
  .trim();
const showDetailsSkeleton = detailsLoading || (!property && loading);
```

How it works

- `fetchPropertyBySlug(slug)` loads the property; the page also resolves a fallback from the list, related, or featured caches.
- While loading it shows `PropertyDetailsSkeleton`; when nothing resolves it shows a "Property Not Found" panel.
- Shortlisting requires authentication. Guests get a toast with a "Log in" action.
- Sharing builds a text summary and uses `navigator.share`, preferring a video file, then the cover image, then plain text; it falls back to clipboard copy.
- Gallery items are the images plus an optional video. The mosaic uses up to four images, and a Radix dialog supports grid/viewer modes with arrow-key navigation and YouTube embedding.
- Specifications are limited to six, special notes to three, and amenities/rules/bills use `FeatureSection` with modals for the full lists.
- Booking cards and the mobile sticky bar offer "Book a visit", optional online payment, and inquiry actions.
- Payment success opens the rating modal only when `forms.features.ratings` is enabled.
- Related properties are fetched with `fetchRelatedProperties(property, sameAgentOnly, page, limit)`; "Show all" switches to page 1 with a limit of 100.
- The map section renders only when both latitude and longitude are non-zero.

Gotchas

- The fallback `cachedProperty ?? currentProperty` can render `currentProperty` even when its slug does not match the route.
- Body overflow is locked for the inspection, payment, and inquiry modals, but not for the gallery, rating, or report modals.
- `handleShortlist` shows the success toast before the async toggle result is known.
- `canReport` requires both `property.agentId` and `"report"` in `forms.features.requests`.

---

8. Properties: shared components

`src/features/properties/components/PropertyCard.tsx` — one listing card (405 lines)

Purpose: Renders a single property with an image carousel, shortlist/share actions, a location link, key facts, and the price. Clicking the card navigates to the detail page.

```
const handleShare = async (e: React.MouseEvent) => {
  e.preventDefault();
  e.stopPropagation();
  const url = `${window.location.origin}/properties/${propertySlug}`;
  const address = addressOf(property);
  const coverImageUrl = property.img || images?.[0] || "";
  const shareText = [
    property.name,
    address,
    `Price: ₦${formatCurrency(property.pricing.total)}${property.duration ? ` ${property.duration.replace(/^\d+\s+/,
 "per ")}` : ""}`,
    `Category: ${property.category}\nURL: ${url}`,
  ]
    .filter(Boolean)
    .join("\n\n");
  // ...
};
```

How it works

- Uses `property.images` when present, otherwise `[property.img]`.
- Supports mouse buttons, dot pagination (up to four visible dots), and touch swipe with a 40px threshold.
- A swipe sets `justSwipedRef`, so the following card click does not navigate.
- Share uses `navigator.share`; it tries to attach the cover image as a `File`, falling back to text share or clipboard copy.
- Shortlist checks `useAuthStore.getState().isAuthenticated`; guests get a toast action linking to `/auth/login`.
- The location link opens Google Maps with coordinates when valid, otherwise the encoded address.
- The description is HTML-stripped and limited to seven words, falling back to `Near ${property.location.nearby}`.
- The listing-type badge colour is chosen by a regex against `property.listingType`.

`src/features/properties/components/PropertySection.tsx` — featured properties (85 lines)

Purpose: The "Featured properties" strip on the home page. It loads featured listings from the store and shows up to four.

```
currentProperties.map((item, i) => (
  <div key={item.id} className={`${i === 3 ? hideOnMdOnly : ""} h-full`}>
    <PropertyCard property={item} />
  </div>
))
```

How it works

- Calls `fetchFeaturedProperties()` on mount and slices the first four.
- Shows four `PropertyCardSkeleton`s while loading, an error panel with retry, or the cards.
- The fourth card is hidden on some breakpoints via `hidden sm:block md:hidden xl:block`.
- "View all" links to `/properties`; the desktop link is hidden below `sm`, with a separate mobile link at the bottom.

`src/features/properties/components/PropertyMap.tsx` — Leaflet map (328 lines)

Purpose: Shows property markers with price or house-pin variants, popups, hover/select callbacks, and a fullscreen mode.

```
function FitBounds({ points, pointsKey }: { points: [number, number][]; pointsKey: string }) {
  const map = useMap();
  const lastFitKeyRef = useRef<string | null>(null);
```

```
  useEffect(() => {
    if (points.length === 0) return;
    if (lastFitKeyRef.current === pointsKey) return; // already fit for this exact set of points

    lastFitKeyRef.current = pointsKey;

    if (points.length === 1) {
      map.setView(points[0], 12);
    } else {
      map.fitBounds(L.latLngBounds(points), { padding: [40, 40] });
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [pointsKey, map]);

  return null;
}
```

How it works

- Filters out properties without valid, non-zero coordinates.
- Builds a stable `pointsKey` from property IDs and coordinates so `FitBounds` only refits when the point set changes.
- Markers fire `onHoverProperty` on hover and `onSelectProperty` on click; popups render `PropertyPreview`.
- Fullscreen portals into `document.body`, locks body scroll, exits on Escape, and renders a placeholder to preserve layout.
- Price markers abbreviate values (`B`, `M`, `K`); property markers use a custom SVG house pin.

`src/features/properties/components/PropertyFiltersModal.tsx` — filter sheet (335 lines)

Purpose: Presentational filter sheet covering location, category, listing type, price, bedrooms, and shared-room-only.

```
<div className="flex items-center justify-between py-3 border-b border-gray-200 dark:border-gray-800">
  <span className="text-gray-900 dark:text-white text-sm">Bedrooms</span>
  <div className="flex items-center gap-4">
    <button
      onClick={() => setBedroomCount?.(Math.max(0, bedroomCount - 1))}
      disabled={bedroomCount === 0}
      className="w-8 h-8 rounded-full border border-gray-600/70 bg-gray-300 dark:border-gray-600 dark:bg-transparent
 flex items-center justify-center text-gray-900 dark:text-white disabled:opacity-30 disabled:cursor-not-allowed hove
r:border-black dark:hover:border-white cursor-pointer"
    >
      –
    </button>
    <span className="text-gray-900 dark:text-white text-sm w-14 text-center">
      {bedroomCount === 0
        ? "Any"
        : bedroomCount >= MAX_BEDROOM_STEP
        ? `${MAX_BEDROOM_STEP}+`
        : bedroomCount}
    </span>
  </div>
</div>
```

How it works

- Visibility is CSS-driven: the modal stays mounted and toggles opacity, pointer events, and translation.
- Backdrop click closes; mobile renders a bottom sheet, desktop a centred panel.
- Location dropdowns are disabled until prerequisites are selected (`state`, then `city`); "Landmark" uses `universityOptions`.
- Category and listing type render "Any" plus option pills, filtering out empty values.
- The bedroom stepper clamps between 0 and 7; 0 shows "Any" and 7 shows "7+".
- "Show X places" closes the modal and pluralises using `totalProperties`.

`src/features/properties/components/FeatureSection.tsx` — feature grid (47 lines)

Purpose: Renders a titled grid of property features with an optional "Show all" action.

```
if (items.length === 0 && extraItems.length === 0) return null;

const hasMore = items.length > limit || extraItems.length > 0;

return (
  <section className="border-t border-gray-200 py-7 first:border-t-0 first:pt-0 dark:border-white/10">
    <h2 className="mb-4 text-xl font-semibold tracking-tight text-gray-900 dark:text-white">
      {title}
    </h2>
    <ul className="grid grid-cols-2 gap-3 sm:grid-cols-3">
      {items.slice(0, limit).map((item, index) => (
        <PropertyFeatureCard key={`${item}-${index}`} label={item} variant={variant} />
      ))}
    </ul>
  </section>
);
```

How it works

- Returns `null` when both lists are empty.
- Only `items.slice(0, limit)` is rendered.
- "Show all" appears when there are more items or when `extraItems` exist.
- `extraItems` are not rendered directly; they only trigger the button.

`src/features/properties/components/PropertyFeatureCard.tsx` — feature card (37 lines)

Purpose: One feature as a card with a resolved Lucide icon and optional prohibited styling.

```
const feature = getPropertyFeatureItem(label);
const Icon =
  feature?.icon ??
  (variant === "rule" ? ShieldCheck : variant === "bill" ? Receipt : House);
const isProhibited = variant === "rule" && (feature?.prohibited ?? false);
```

How it works

- Resolves an icon through the feature registry.
- Falls back to `ShieldCheck` for rules, `Receipt` for bills, otherwise `House`.
- Prohibited features use red borders/background and a `Ban` badge over the icon.
- `prohibited` only applies to the `rule` variant.

`src/features/properties/components/PropertyFeatureItem.tsx` — feature list item (61 lines)

Purpose: A compact feature row with optional divider, highlighting, and icon resolution.

```
const feature = getPropertyFeatureItem(label);
const fallbackIcon =
  variant === "rule" ? ShieldCheck : variant === "bill" ? Receipt : showIcon ? House : null;
const Icon = icon ?? feature?.icon ?? fallbackIcon;
const isProhibited = variant === "rule" && (feature?.prohibited ?? false);
```

How it works

- Uses an explicit `icon` prop first, then the registry icon, then a variant fallback.
- The default fallback is `null` unless `showIcon` is true; rules and bills always get an icon.
- `divided` adds a bottom border except on the last item.
- `highlighted` adds a purple left border/background and fills the icon.
- Prohibited rules use a red icon and a small `Ban` badge.

`src/features/properties/components/propertyFeatureIcons.ts` — icon registry (187 lines)

Purpose: The canonical feature/amenity registry, grouped into "What this place offers", "House rules", and "Bills & fees", with normalization and aliases for API labels.

```
const normalizeFeatureLabel = (label: string) =>
  label.toLowerCase().replace(/[^a-z0-9]/g, "");

const propertyFeatureByLabel = new Map(
  propertyFeatureIconGroups.flatMap((group) =>
    group.items.map((item) => [normalizeFeatureLabel(item.label), item]),
  ),
);

const featureLabelAliases = new Map([
  ["marriedcoupleonly", "marriedcouplesonly"],
  ["24hourelectricity", "24hourelectricitysupply"],
  ["prepaidmeter", "personalprepaidmeter"],
  ["nopets", "nopetsallowed"],
  // ...
]);
```

How it works

- Creates two custom Lucide icons (`ShowerCubicle`, `BuiltInKitchen`) with `createLucideIcon`.
- Builds a lookup map at module load, normalizing labels by lowercasing and removing non-alphanumerics.
- `getPropertyFeatureItem` normalizes the input, applies an alias, then returns the matching item.
- `getPropertyFeatureIcon` returns only the icon or `undefined`.
- Unknown labels return `undefined`, so consumers must provide fallbacks.

`src/features/properties/components/CustomDropdown.tsx` — controlled dropdown (112 lines)

Purpose: A reusable controlled dropdown used across filters and modals.

```
<button
  type="button"
  disabled={disabled}
  onClick={() => {
    if (!disabled) {
      setOpen(!open);
    }
  }}
  className={`${buttonClassName || defaultButtonClass} flex items-center justify-between ${
    disabled ? "cursor-not-allowed pointer-events-none" : "cursor-pointer"
  }`}
>
  <div className="flex items-center gap-2 min-w-0 flex-1">
    {icon}
    <span className={`truncate whitespace-nowrap ${!isSelected ? "opacity-80" : ""}`}>
      {selectedLabel}
    </span>
  </div>
</button>
```

How it works

- Tracks open state locally and closes on an outside `mousedown`.
- Closes automatically when `disabled` becomes true.
- Shows the selected label or the placeholder; empty-value selections count as unselected.
- Option click calls `onChange(value)` and closes the menu.
- The menu stays mounted with opacity/pointer transitions and a high z-index.
- No keyboard navigation, Escape handling, or ARIA roles are implemented.

---

9. Properties: modals and forms

`src/features/properties/components/PropertyPaymentModal.tsx` — buying a property (429 lines)

Purpose: Collects guest/customer details, holds the property through the API, launches payment or shows bank-transfer instructions, and optionally triggers the rating flow.

```
try {
  // 1. Hold a place. SabiFlow works out what's owed from the listing.
  const purchase = await propertiesAPI.purchase(
    property.id,
    { name: fullName.trim(), email: email.trim(), phone: phone.trim(), consent: agreed },
    requestKey.current(),
  );
  noteTestMode(purchase);
  if (purchase.test) return;
  if (purchase.data.holdUntil) setHeldUntil(new Date(purchase.data.holdUntil).getTime());

  // Above the online limit: show where to send it, and the team confirms it
  if (purchase.bankTransfer) {
    setBankTransfer(purchase);
    finish();
    return;
  }
  // ...
}
```

How it works

- A Radix dialog with `modal={false}` and a responsive bottom-sheet/centred layout.
- Persists guest name/email/phone/consent in the `rewaciti_payment_modal` cookie. Signed-in users load their profile and sync a changed phone number into the auth store.
- Submits through `propertiesAPI.purchase` with a `useRequestKey` idempotency key; `noteTestMode` short-circuits test responses.
- If the response includes `bankTransfer`, it shows the amount and details instead of the form.
- Otherwise it finds the first unpaid payment and calls `payInvoice`.
- Handles `paid`, `pending`, `cancelled`, and `failed`; only `paid` calls `onPaymentSuccess`.
- `finish()` refetches the property, renews the request key, clears the hold timer, and clears the cookie.
- `byBankTransfer` changes the submit label when `price > onlineLimit`; the actual transfer decision comes from the API.

`src/features/properties/components/InquiryModal.tsx` — ask about a property (260 lines)

Purpose: Sends a property inquiry with contact details and a message.

```
try {
  // SabiFlow links the property, its agent and the customer's record
  const reply = await requestsAPI.inquiry(
    {
      propertyId: property.id,
      name: fullName.trim(),
      email: email.trim(),
      phone: phone.trim(),
      message: message.trim(),
      consent: agreed,
    },
    requestKey.current(),
  );
  requestKey.renew();
  noteTestMode(reply);
  // ...
}
```

How it works

- Loads saved form data from the `rewaciti_property_details_inquiry` cookie.
- When opened for a signed-in user, fetches the latest profile and fills name/email/phone.
- Writes form state to the cookie on changes, skipping the first render via `cookieLoaded`.
- Validates consent and required contact fields before submitting.
- On success it toasts, clears the fields, writes an empty cookie, and closes.
- Submit is disabled unless consent is checked and name/email/phone are non-empty.
- The selected-property field is read-only and combines the name and `addressOf(property)`.

`src/features/properties/components/ReportAgentModal.tsx` — report an agent (318 lines)

Purpose: Lets users report an agent for a property, with a reason, description, contact details, and consent.

```
try {
  // The report goes on the agent's record, alongside the listing
  const reply = await requestsAPI.agentReport(
    {
      propertyId: property.id,
      reason,
      message: description.trim(),
      name: name.trim(),
      email: email.trim(),
      phone: phone.trim(),
      consent: agreed,
    },
    requestKey.current(),
  );
  requestKey.renew();
  noteTestMode(reply);
  // ...
}
```

How it works

- Reads report reasons from `usePropertyStore` (`forms?.reportReasons`).
- Persists guest details, reason, description, and consent in `rewaciti_report_modal`.
- Older saved reasons are normalized with `.replace("-", "_")`.
- Signed-in users see their customer details and the latest phone number.
- Validates contact info, reason/description, and consent before submitting.
- On success it toasts, closes, clears the reason/description, and writes an empty cookie.

`src/features/properties/components/ServiceRatingModal.tsx` — rating + Trustpilot (174 lines)

Purpose: Post-payment rating modal that submits a 1–5 star deal rating and sends a Trustpilot invitation.

```
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  if (rating === 0) {
    toast.error("Please select a rating.");
    return;
  }

  setIsSubmitting(true);

  try {
    if (!userData) throw new Error("Nothing to rate");
    // Counts towards the property's rating and its agent's
    const reply = await requestsAPI.rating({
      type: "deal",
      reference: userData.dealNumber,
      email: userData.email,
      stars: rating,
      comment: feedback.trim() || undefined,
    });
    noteTestMode(reply);
```

```
    sendTrustpilotInvitation();
    // ...
  }
};
```

How it works

- Stars update `rating` on click and use `hoverRating` for preview.
- Submit requires a non-zero rating and `userData`.
- Sends the rating as `type: "deal"` with the deal number and email.
- Sends the Trustpilot invitation for every submission, regardless of rating, to avoid review gating.
- Both success and error close the modal and navigate to `/properties`.

`src/features/inspections/components/BookInspectionModal.tsx` — book a visit (472 lines)

Purpose: A Radix dialog that collects guest/auth details, loads available slots, books an inspection, and handles the visit fee.

```
try {
  // 1. Book the time. It's held while the fee is paid.
  const reply = await visitsAPI.book(
    {
      propertyId: property.id,
      name: fullName.trim(),
      email: bookingEmail.trim(),
      phone: phone.trim(),
      date: visitDate,
      start: visitTime,
      mode,
      consent: agreed,
    },
    requestKey.current(),
  );
  noteTestMode(reply);
  if (reply.test) return;

  const booking = reply.data;
  if (booking.holdExpiresAt) setHeldUntil(new Date(booking.holdExpiresAt).getTime());
  // ...
}
```

How it works

- Uses `useAuthStore` for the signed-in customer and `usePropertyStore` for `forms.visit` rules.
- Persists guest name/email/phone/consent in the `rewaciti_inspection_modal` cookie.
- When opened for a signed-in user, fetches the profile and syncs the phone into the auth store.
- Computes the earliest visit date from `minNoticeHours` and loads slots from `visitsAPI.slots(property.id, visitDate)` whenever the date changes.
- Only `slot.available` times are selectable; changing the date clears the selected time.
- Validates name, email, phone, date/time, and consent, then calls `visitsAPI.book`.
- If the booking is `pending_payment` with an `invoiceId`, calls `payInvoice` and handles `paid`, `pending`, `cancelled`, and `failed`.
- The fee comes from `property.visitationfee`: `null` → "Loading...", `0` → "Free", otherwise naira-formatted.

---

10. Properties: request, co-rent, and list pages

`src/features/properties/pages/RequestPropertyPage.tsx` — what are you looking for? (440 lines)

Purpose: A property-request form. The team matches it against future listings and emails the customer.

```
const city = preferredCity.trim();
const knownCity = cities.find((item) => item.city.toLowerCase() === city.toLowerCase());
setIsSubmitting(true);
try {
  const reply = await requestsAPI.propertyRequest(
    {
      name: name.trim(),
      email: email.trim(),
      phone: phone.trim(),
      consent: agreed,
      purpose: purpose || undefined,
      typeId: category || undefined,
      state: knownCity?.state,
      city: knownCity?.city ?? city,
      nearby: nearby.trim() || undefined,
      bedrooms: bedrooms ? Number(bedrooms) : undefined,
      maxBudget: maxBudget ? Number(maxBudget) : undefined,
      moveIn: moveIn.trim() || undefined,
      message: message.trim() || undefined,
    },
    requestKey.current(),
  );
  // ...
}
```

How it works

- Purpose options come from `PURPOSE_LABELS`; category options come from the store.
- Cities are flattened from the location hierarchy; landmarks are de-duplicated and sorted.
- City and landmark inputs use HTML datalists for suggestions.
- Drafts live in the `rewaciti_property_inquiry` cookie, including legacy keys.
- Signed-in users are prefilled from `authAPI.getProfile()` and the auth store, guarded by an `active` flag.
- `roomCountsDisabled` is true for `selfcontain`, `singleroom*`, and `land*` categories; bedrooms are cleared when disabled.
- Submission requires consent and non-empty name, email, phone, and preferred city.
- If the city matches a known city case-insensitively, its state is sent too; otherwise only the typed city is sent.
- On success the cookie is cleared, a toast is shown, and the user is navigated to `/properties`.

`src/features/properties/pages/CoRentPage.tsx` — find someone to share with (452 lines)

Purpose: A co-renting request form that collects contact details, location, sharing preferences, budget, and move-in timing.

```
try {
  // The team matches people asking for the same kind of place
  const reply = await requestsAPI.coRent(
    {
      name: name.trim(),
      email: email.trim(),
      phone: phone.trim(),
      consent: agreed,
      location: preferedLocation.trim(),
      nearby: nearby.trim() || undefined,
      gender: (gender || undefined) as CoRentRequest["gender"],
      partnerGender: preferredPartnerGender as CoRentRequest["partnerGender"],
      sharing: sharingOf(roomType),
      budgetPerPerson: budget ? Number(budget) : undefined,
      moveIn: moveInTimeline.trim() || undefined,
      message: [roomType && `Share type: ${roomType}`, message.trim()].filter(Boolean).join("\n\n") || undefined,
    },
    requestKey.current(),
```

```
  );
  // ...
}
```

How it works

- Draft data is read from `rewaciti_corent_request`; malformed JSON is ignored.
- Saved gender values are lowercased for compatibility with older drafts, and budget is reduced to digits.
- Signed-in users get name/email/phone from the profile and auth store; a changed phone is written back to the store.
- The draft is saved on every change except the initial render.
- Submission requires consent and non-empty name, email, phone, and preferred location.
- Room type is sent inside the message as `Share type: ...`; `sharing` is derived from it.
- `requestKey` is renewed after submission and `noteTestMode` handles test-mode feedback.
- On success the form resets and the cookie is overwritten with empty values.

`src/features/properties/pages/ListPropertyPage.tsx` — become a lister (376 lines)

Purpose: An application form for agents, owners, caretakers, and developers who want to list properties with RewaCiti.

```
try {
  // Approving it in SabiFlow invites them to the agent portal
  const reply = await requestsAPI.agentApplication(
    {
      name: name.trim(),
      email: email.trim(),
      phone: phone.trim(),
      consent: agreed,
      role: LISTER_ROLE_OPTIONS.find((option) => option.value === listerRole)?.label ?? listerRole,
      location: location.trim(),
      message: [
        `Listing purpose: ${LISTING_PURPOSE_OPTIONS.find((option) => option.value === listingPurpose)?.label ?? list
ingPurpose}`,
        message.trim(),
      ].filter(Boolean).join("\n\n"),
    },
    requestKey.current(),
  );
  // ...
}
```

How it works

- The draft cookie is `rewaciti_become_lister_inquiry`.
- Signed-in users get name/email/phone from the profile and auth store.
- Submission requires consent and non-empty name, email, phone, and location.
- The API receives the role label, not the option value.
- Listing purpose is sent only inside the message.
- On success the form resets and the cookie is cleared; the page does not navigate away.
- The submit button is disabled unless the form is valid, which also requires a listing purpose.

`src/features/properties/pages/PropertyFeatureIconsPage.tsx` — icon reference (84 lines)

Purpose: A static reference page showing every feature/amenity icon group, including prohibited-rule badges.

```
{propertyFeatureIconGroups.map((group) => {
  const headingId = group.title
    .toLowerCase()
    .replace(/[^a-z0-9]+/g, "-");

  return (
    <section key={group.title} aria-labelledby={headingId}>
      <div className="mb-5">
        <h2 id={headingId} className="text-2xl font-semibold text-gray-900 dark:text-white">
```

```
          {group.title}
        </h2>
        <p className="mt-1 text-sm text-gray-600 dark:text-gray-400">
          {group.description}
        </p>
      </div>
      <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4">
        {group.items.map((item) => (
          <FeatureIconCard key={item.label} item={item} />
        ))}
      </div>
    </section>
  );
})}
```

How it works

- No state, effects, or data fetching: it renders the static registry.
- `FeatureIconCard` shows one icon and label; prohibited items get a red circular ban badge.
- Heading ids are derived from the group title.
- The grid is one column on mobile, two on small, three on large, and four on extra-large.
- `Navbar` and `Footer` wrap the reference content.

---

11. Blog, comments, and FAQ

These three features follow the same pattern: a Zustand store loads static JSON from `public/data`, paginates on the client, and pages render the data. They do not use the SabiFlow API.

`src/features/blog/store/useBlogStore.ts` — blog posts (39 lines)

```
fetchPosts: async () => {
  set({ loading: true });
  try {
    const res = await axios.get<BlogPost[]>("/data/Blog.json");
    const sanitizedPosts = res.data.map(post => ({
      ...post,
      image: ensureHttps(post.image)
    }));
    set({ posts: sanitizedPosts, loading: false });
  } catch (err) {
    console.error("Failed to fetch blog posts", err);
    set({ loading: false });
  }
},
```

How it works

- Loads `/data/Blog.json` from `public/data`.
- Sanitises each image URL with `ensureHttps`.
- `getPostBySlug` searches the already-loaded `posts` array; it never fetches a single post.
- `ITEMS_PER_PAGE` is 6; `nextPage` clamps to the last page, `prevPage` clamps at 0.
- There is no error state and no request deduplication.

`src/features/blog/pages/BlogPage.tsx` — blog listing (155 lines)

Purpose: Hero, post grid, skeletons, empty state, and pagination.

```
<div className="grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-3 lg:gap-6">
  {currentPosts.map((post) => (
    <article key={post.id} className="flex flex-col overflow-hidden rounded-2xl border border-gray-900/15 bg-white t
ransition-shadow duration-300 hover:shadow-lg dark:border-white/10 dark:bg-[#1A1A1A]">
      <Link to={`/blog/${post.slug}`} aria-label={`Read ${post.title}`}>
        <img
          src={post.image}
          alt={post.title}
          loading="lazy"
          decoding="async"
          className="h-60 w-full object-cover"
        />
      </Link>
      {/* ... */}
    </article>
  ))}
</div>
```

How it works

- Calls `fetchPosts()` once on mount.
- Computes `totalPages` and slices `currentPosts` from the full list.
- Shows six `BlogCardSkeleton`s while loading and a message when there are no posts.
- Pagination appears only when `totalPages > 1`; buttons disable at the boundaries.
- Each card links to `/blog/${post.slug}`.
- Uses `Helmet` for title, description, Open Graph tags, and the canonical URL.

`src/features/blog/pages/BlogPostDetailsPage.tsx` — single post (192 lines)

Purpose: Renders one post with metadata, lightweight content formatting, tags, and related posts.

```
<div className="prose prose-lg dark:prose-invert max-w-none leading-relaxed text-gray-700 dark:text-gray-300">
  {post.content.split('\n').map((paragraph, index) => {
    if (!paragraph.trim()) return <div key={index} className="h-4" />;

    const parts = paragraph.split(/(\*\*.*?\*\*)/g);
    return (
      <p key={index} className="mb-4">
        {parts.map((part, i) => {
          if (part.startsWith('**') && part.endsWith('**')) {
            return <strong key={i} className="font-bold text-gray-900 dark:text-white">{part.slice(2, -2)}</strong>;
          }
          return part;
        })}
      </p>
    );
  })}
</div>
```

How it works

- Gets `slug` from `useParams`, calls `fetchPosts()`, then looks up the post with `getPostBySlug`.
- Renders a skeleton while loading and "Post not found" when nothing matches.
- Related posts are same-category posts excluding the current one, sliced to three.
- Content is split by newline into paragraphs; only `**bold**` is converted to `<strong>`.
- Metadata includes Open Graph article fields and JSON-LD `BlogPosting`.

`src/features/blog/index.ts` — barrel (3 lines)

```
export { default as BlogPage } from "./pages/BlogPage";
export { default as BlogPostDetailsPage } from "./pages/BlogPostDetailsPage";
export * from "./store/useBlogStore";
```

`src/features/comments/store/useCommentStore.ts` — testimonials (37 lines)

```
fetchComments: async () => {
  set({ loading: true });
  try {
    const res = await axios.get<Comment[]>("/data/Comments.json");
    const sanitizedComments = res.data.map(comment => ({
      ...comment,
      img: ensureHttps(comment.img)
    }));
    set({ comments: sanitizedComments, loading: false });
  } catch (err) {
    console.error("Failed to fetch comments", err);
    set({ loading: false });
  }
},
```

How it works

- Loads `/data/Comments.json` and sanitises each image URL.
- `ITEMS_PER_PAGE` is 3; pagination is client-side.
- There is no error state or retry logic.

`src/features/comments/components/CommentSection.tsx` — testimonial strip (149 lines)

Purpose: A three-card testimonial section with expandable descriptions and pagination.

```
<p className="text-sm leading-6 text-gray-600 dark:text-gray-300">
  {expanded[comment.id]
    ? comment.description
    : truncateWords(comment.description, 10)}
  {isLong && (
    <button
      type="button"
      onClick={() =>
        setExpanded((prev) => ({ ...prev, [comment.id]: !prev[comment.id] }))
      }
      className="ml-2 cursor-pointer font-semibold text-[#703BF7] hover:underline dark:text-[#b79bff]"
    >
      {expanded[comment.id] ? "Show less" : "... Read more"}
    </button>
  )}
</p>
```

How it works

- Fetches comments on mount and reads pagination from the store.
- Shows three `CommentCardSkeleton`s while loading.
- Descriptions are truncated to ten words when collapsed.
- Ratings render as repeated ★ / ☆; the reviewer's location links to Google Maps.
- "View all" links to `/AllComments`.

`src/features/comments/pages/AllCommentsPage.tsx` — all testimonials (116 lines)

Purpose: Renders every comment in a responsive grid, without pagination.

```
const Stars = ({ rating }: { rating: number }) => {
  const filled = Math.max(0, Math.min(5, Math.round(rating)));
  return (
    <div role="img" aria-label={`Rated ${filled} out of 5`} className="flex gap-0.5 text-lg leading-none">
      {[0, 1, 2, 3, 4].map((i) => (
        <span
          key={i}
          aria-hidden="true"
          className={i < filled ? "text-amber-400" : "text-gray-300 dark:text-gray-600"}
        >
          ★
        </span>
      ))}
    </div>
  );
};
```

How it works

- Calls `fetchComments()` on mount.
- Shows six skeletons while loading and an empty state when there are no comments.
- Renders all comments, with clamped star ratings and Google Maps location links.
- Uses `Helmet` with a canonical URL.

`src/features/comments/index.ts` — barrel (3 lines)

```
export { default as CommentSection } from "./components/CommentSection";
export { default as AllCommentsPage } from "./pages/AllCommentsPage";
export * from "./store/useCommentStore";
```

`src/features/faq/store/useFAQStore.ts` — FAQs (32 lines)

```
fetchFAQs: async () => {
  set({ loading: true });
  try {
```

```
    const res = await axios.get<FAQ[]>("/data/FAQ.json");
    set({ faq: res.data, loading: false });
  } catch (err) {
    console.error("Failed to fetch FAQs", err);
    set({ loading: false });
  }
},
```

How it works

- Loads `/data/FAQ.json`.
- `ITEMS_PER_PAGE` is 3; pagination is client-side.
- The state key is `faq` (singular), not `faqs`.
- No URL sanitisation is needed here, unlike blog/comments.

`src/features/faq/components/FAQSection.tsx` — home-page FAQ strip (120 lines)

Purpose: A three-question FAQ section with expandable answers.

```
<div className="grid grid-cols-1 gap-4 md:grid-cols-3 lg:gap-6">
  {loading
    ? [...Array(3)].map((_, i) => <FAQCardSkeleton key={i} />)
    : currentFAQs.map((item) => {
        const isLong = item.answer.split(" ").length > 8;
        return (
          <article
            key={item.id}
            className="flex flex-col rounded-3xl border border-gray-200 bg-white p-6 text-gray-900 dark:border-white
/10 dark:bg-[#1A1A1A] dark:text-white"
          >
            <h3 style={SERIF} className="mb-2 text-lg font-semibold">
              {item.question}
            </h3>

            <p className="text-sm leading-6 text-gray-600 dark:text-gray-300">
              {expanded[item.id] ? item.answer : truncateWords(item.answer, 8)}
              {!expanded[item.id] && isLong && "…"}
            </p>
            {/* ... */}
          </article>
        );
      })}
</div>
```

How it works

- Fetches FAQs on mount and slices three per page.
- Shows three `FAQCardSkeleton`s while loading.
- Answers over eight words are truncated with a "Read more" button.
- "View all" links to `/AllFAQs`.

`src/features/faq/pages/AllFAQPage.tsx` — all FAQs (77 lines)

Purpose: Renders every question and answer in a responsive grid.

```
<div className="grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-3 lg:gap-6">
  {loading ? (
    [...Array(6)].map((_, i) => <FAQCardSkeleton key={i} />)
  ) : faq.length === 0 ? (
    <p className="col-span-full rounded-2xl border border-dashed border-gray-300 bg-white p-10 text-center text-sm t
ext-gray-600 dark:border-white/15 dark:bg-[#1A1A1A] dark:text-gray-400">
      No questions have been added yet. Check back soon.
    </p>
  ) : (
    faq.map((item: FAQ) => (
      <article
        key={item.id}
```

```
        className="rounded-2xl border border-gray-200 bg-white p-6 text-gray-900 shadow-sm transition-colors hover:b
order-[#703BF7]/50 dark:border-white/10 dark:bg-[#1A1A1A] dark:text-white dark:hover:border-[#a98bff]/50"
      >
        <h2 className="mb-2 text-lg font-semibold leading-snug">
          {item.question}
        </h2>
        <p className="text-sm leading-relaxed text-gray-600 dark:text-gray-400">
          {item.answer}
        </p>
      </article>
    ))
  )}
</div>
```

How it works

- Calls `fetchFAQs()` on mount.
- Shows six skeletons while loading and an empty state when there are none.
- Renders all FAQs with no pagination or expand/collapse.
- Uses `Helmet` with a canonical URL.

`src/features/faq/index.ts` — barrel (3 lines)

```
export { default as FAQSection } from "./components/FAQSection";
export { default as AllFAQPage } from "./pages/AllFAQPage";
export * from "./store/useFAQStore";
```

---

12. Layout: Navbar and Footer

`src/shared/components/Layout/Navbar.tsx` — navigation header (318 lines)

Purpose: App-wide navigation with a promo bar, desktop/mobile navigation, theme toggle, auth/profile actions, and a saved-properties entry.

```
<div
  className="w-full bg-gray-400 text-gray-900 dark:bg-[#1A1A1A] dark:text-white"
  style={{
    backgroundImage: "url('/logo/Abstract Design.png')",
    backgroundPosition: "center",
  }}
>
  <div className={`${PAGE_CONTAINER} grid h-10 grid-cols-[1fr_auto_1fr] items-center`}>
    <div aria-hidden="true" />
    <div className="flex items-center justify-center text-sm">
      <p className="mr-1 flex items-center">✨ Find your dream property</p>
      <NavLink
        to="/blog"
        aria-label="View blog page"
        className="hidden font-semibold underline underline-offset-2 sm:block"
      >
        View Blog
      </NavLink>
    </div>
    {/* ... */}
  </div>
</div>
```

How it works

- Reads theme, auth, and property state from the Zustand stores.
- Computes the active nav item, profile initials, and saved-property count.
- Handles the mobile menu (body scroll lock, Escape, resize-to-desktop, route changes), logout, and navigation.
- The top bar scrolls away; the main header is sticky.
- The mobile drawer slides from the right with a backdrop.
- Imports `NavLink`, `useNavigate`, and `useLocation` from `react-router` (not `react-router-dom`).
- The saved-properties button prompts a login toast when signed out; signed in, it navigates to `/auth/profile?tab=saved`.
- `/studentarea` is treated as part of Properties for active-state purposes.

`src/shared/components/Layout/Footer.tsx` — site footer (213 lines)

Purpose: Branding, newsletter subscription, navigation/contact links, social links, and the legal bar.

```
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();

  if (!email) {
    toast.error("Please enter your email");
    return;
  }

  setIsSubmitting(true);

  const payload = {
    companyId: COMPANY_ID,
    pipelineId: "69cec882dd40685bfe20ad43",
    title: `Newsletter Subscription: ${email}`,
    name: "Newsletter Subscriber",
    email: email,
    note: "User subscribed to newsletter via footer form.",
  };

  try {
```

```
    await axios.post("https://api.sabiflow.com/api/crm/deals/guest", payload);
    toast.success("Subscribed successfully!");
    setEmail("");
  } catch (error) {
    console.error("Error subscribing:", error);
    toast.error("Failed to subscribe. Please try again.");
  } finally {
    setIsSubmitting(false);
  }
};
```

How it works

- Manages newsletter email/submission state.
- Posts to SabiFlow's CRM guest-deals endpoint with `COMPANY_ID` and a hard-coded `pipelineId`.
- Renders a responsive grid and a bottom legal bar with the current year.

`src/shared/components/Layout/index.ts` — barrel (2 lines)

```
export { default as Navbar } from "./Navbar";
export { default as Footer } from "./Footer";
```

---

13. Marketing and legal pages

`src/app/pages/Home.tsx` — landing page (260 lines)

Purpose: Composes SEO metadata, a hero search, service highlights, featured properties, testimonials, and FAQs.

```
const handleSearch = (e: React.FormEvent) => {
  e.preventDefault();
  const params = new URLSearchParams();
  if (tab !== "all") params.set("listing", tab);
  if (area) params.set("area", area);
  if (type) params.set("category", type);
  if (budget) params.set("maxPrice", budget);
  const query = params.toString();
  navigate(`/Properties${query ? `?${query}` : ""}`);
};
```

How it works

- Uses `useScrollToHash` and local search state (`tab`, `area`, `type`, `budget`).
- Fetches area options from `usePropertyStore.getState().fetchLocations()` on mount.
- Search submits to `/Properties` with query parameters.
- Renders `PropertySection`, `CommentSection`, and `FAQSection` below the hero.
- Budget strips non-digits and stores raw digits.

`src/app/pages/About.tsx` — company story (503 lines)

Purpose: Story, values, achievements, process steps, and team contact interactions.

```
const handleMessage = (
  e: FormEvent<HTMLFormElement>,
  member: TeamMember,
) => {
  e.preventDefault();

  const form = e.currentTarget;
  const input = form.elements.namedItem("message") as HTMLInputElement;

  const subject = `Message for ${member.name}`;
  const body = [
    `Hello ${member.name},`,
    "",
    input.value,
    "",
    "Best regards,",
  ].join("\n");

  window.location.href = `mailto:${member.email}?subject=${encodeURIComponent(
    subject,
  )}&body=${encodeURIComponent(body)}`;

  input.value = "";
};
```

How it works

- Uses `useScrollToHash` and `Helmet`.
- Sections are anchored: `Journey`, `Values`, `Achievements`, `Process`, `Team`.
- `StepCard` expands/collapses long descriptions.
- Team cards link to LinkedIn and use a `mailto:` form for messages.
- Team and content data are static constants in the file.

`src/app/pages/Service.tsx` — services (372 lines)

Purpose: Describes the offerings (selling, management, investing) with overview cards and calls to action.

```
const ServiceSection = ({
  id,
  eyebrow,
  title,
  intro,
  items,
  cta,
  photo,
  reverse,
  tone,
}: SectionConfig & { tone: string }) => (
  <section id={id} className={`scroll-mt-24 py-14 sm:py-20 ${tone}`}>
    <div className={PAGE_CONTAINER}>
      <div className="max-w-2xl">
        <p className="text-sm font-semibold text-[#703BF7] dark:text-[#b79bff]">{eyebrow}</p>
        <h2 style={SERIF} className="mt-2 text-3xl font-semibold text-gray-900 dark:text-white sm:text-4xl">
          {title}
        </h2>
        <p className="mt-4 leading-7 text-gray-700 dark:text-gray-400">{intro}</p>
      </div>
      {/* ... */}
    </div>
  </section>
);
```

How it works

- Uses `useScrollToHash` and `Helmet`.
- Overview cards link to `/properties` or in-page hashes.
- Detailed sections alternate tones and can reverse the photo/CTA layout.
- The page ends with a closing CTA linking to contact and properties.

`src/app/pages/Contact.tsx` — contact form (545 lines)

Purpose: Quick contact cards, an inquiry form with draft persistence, and office/map details.

```
try {
  const reply = await requestsAPI.contact(
    {
      name: fullName,
      email: email.trim(),
      phone: phone.trim(),
      consent: agreed,
      message: message.trim(),
      topic: labelOf(INQUIRY_TYPE_OPTIONS, inquiryType),
      heardFrom: labelOf(SOURCE_OPTIONS, source),
    },
    requestKey.current(),
  );
  requestKey.renew();
  noteTestMode(reply);
  toast.success(
    <div className="whitespace-pre-wrap">
      Message sent successfully!
      <br />A member of our team will get back to you soon.
    </div>,
  );
  // ...
}
```

How it works

- Prefills from a cookie and/or the signed-in customer's profile (`authAPI.getProfile()`).
- Persists form state to a cookie, skipping the first render.
- Validates consent and core identity/message fields before submitting.
- Submits through `requestsAPI.contact` with an idempotency key.
- On success it renews the key, shows a toast, and resets the form.

- Renders office details and an embedded Google map.

`src/app/pages/Terms.tsx` — Terms of Use (270 lines)

Purpose: Static legal page with numbered sections and SEO metadata.

```
<section className="bg-white/90 dark:bg-[#1A1A1A] border rounded-xl p-6 mb-4">
  <h2 className="text-xl font-semibold mb-2">
    5. Property Visits & Payments
  </h2>

  <p>
    Each agent sets their own visitation fee. Any visitation fee
    charged covers the agent&apos;s time and is normally paid once per
    agreed visitation session, not per property visited. Applicable
    fees will be shown at the point of booking.
  </p>

  <p className="mt-2">
    A separate <strong>₦500 RewaCiti service charge</strong> is added
    to the agent&apos;s visitation fee. This charge is non-refundable,
    including if a problem occurs with the booking or visitation.
  </p>
</section>
```

How it works

- Renders `Navbar`, Helmet metadata, styled static sections, and `Footer`.
- Content is hard-coded JSX, so updates require code edits.
- The canonical URL is `https://rewaciti.com/terms`.

`src/app/pages/PrivacyPolicy.tsx` — Privacy Policy (211 lines)

Purpose: Static privacy page describing data collection, use, and rights.

```
<section className="bg-white/90 dark:bg-[#1A1A1A] border rounded-xl p-6 mb-4">
  <h2 className="text-xl font-semibold mb-2">
    8. Cookies & Tracking Technologies
  </h2>
  <p>
    Cookies and similar technologies are used to enhance functionality,
    store preferences, and analyze user behavior. Users may manage cookie
    preferences through their browser settings.
  </p>
</section>
```

How it works

- Renders `Navbar`, Helmet metadata, numbered policy sections, and `Footer`.
- All text is hard-coded.
- The canonical URL is `https://rewaciti.com/privacy-policy`.

`src/app/pages/NotFound.tsx` — 404 page (42 lines)

Purpose: Unknown-route page with a link back home.

```
<Helmet>
  <title>404 Page Not Found | RewaCiti</title>
  <meta name="description" content="The page you're looking for doesn't exist. Return to the RewaCiti homepage to ex
plore properties, student housing, and services." />
  <meta property="og:title" content="404 Page Not Found | RewaCiti" />
  <meta property="og:description" content="This link is broken or the page has moved. Visit RewaCiti to find propert
y listings, student housing, and real estate services." />
  <meta property="og:type" content="website" />
  <meta name="twitter:card" content="summary_large_image" />
  <link rel="canonical" href="https://rewaciti.com/404" />
</Helmet>
```

How it works

- Sets 404 metadata, renders the layout chrome, and centres the error message.
- The canonical URL points to `/404`.

---

14. Build, config, and deployment

`package.json` — scripts and dependencies

```
"scripts": {
  "dev": "vite",
  "build": "tsc -b && vite build",
  "lint": "eslint .",
  "preview": "vite preview"
}
```

How it works

- `build` type-checks with `tsc -b`, then bundles with Vite.
- `vite` is aliased to `npm:rolldown-vite@7.1.14` in both `devDependencies` and `overrides`.
- Runtime: React 18, React Router 7, Zustand 5, Axios, Radix UI, Tailwind v4, Leaflet, Paystack, Sonner, React Helmet Async.
- React packages are pinned to `18.0.0`, while `@types/react*` are v19 in devDependencies.

`vite.config.ts` — Vite setup

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'
import tsconfigPaths from 'vite-tsconfig-paths'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), tailwindcss(), tsconfigPaths()],
  server: {
    open: true,
  },
})
```

How it works

- Enables React, Tailwind v4, and TypeScript path resolution.
- Opens the browser when the dev server starts.

`tsconfig.app.json` — TypeScript

```
"target": "ES2022",
"useDefineForClassFields": true,
"lib": ["ES2022", "DOM", "DOM.Iterable"],
"module": "ESNext",
"types": ["vite/client"],
"skipLibCheck": true,
"moduleResolution": "bundler",
"allowImportingTsExtensions": true,
"verbatimModuleSyntax": true,
"moduleDetection": "force",
"noEmit": true,
"jsx": "react-jsx",
"strict": true,
"noUnusedLocals": true,
"noUnusedParameters": true,
"erasableSyntaxOnly": true,
"noFallthroughCasesInSwitch": true,
"noUncheckedSideEffectImports": true
```

How it works

- Bundler-mode TypeScript with strict checks and no emit.

- `noUnusedLocals` / `noUnusedParameters` make unused code a build error.
- `verbatimModuleSyntax` and `erasableSyntaxOnly` constrain import and syntax style.

`eslint.config.js` — linting

```
export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{ts,tsx}'],
    extends: [
      js.configs.recommended,
      tseslint.configs.recommended,
      reactHooks.configs['recommended-latest'],
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      ecmaVersion: 2020,
      globals: globals.browser,
    },
    rules: {
      '@typescript-eslint/no-unused-vars': [
        'error',
        { varsIgnorePattern: '^_', argsIgnorePattern: '^_' },
      ],
    },
  },
])
```

How it works

- Flat config API with recommended JS/TS, React Hooks, and React Refresh presets.
- Variables and arguments prefixed with `_` are allowed to be unused.
- `dist` is ignored.

`src/app/styles/App.css` — global styles (384 lines)

```
@import "tailwindcss";
@import "tw-animate-css";

@custom-variant dark (&:is(.dark *));

@theme inline {
  --font-sans: "DM Sans", ui-sans-serif, system-ui, sans-serif;
  --font-serif: "DM Serif Display", Georgia, serif;
  --radius-sm: calc(var(--radius) - 4px);
  --radius-md: calc(var(--radius) - 2px);
  --radius-lg: var(--radius);
  --radius-xl: calc(var(--radius) + 4px);
  --color-background: var(--background);
  --color-foreground: var(--foreground);
}
```

How it works

- Imports Tailwind and `tw-animate-css`.
- Declares the class-based `dark` variant.
- Defines design tokens (fonts, radii, semantic colours) in `@theme inline`.
- Contains light/dark CSS variables, base layer rules, and custom animations for dialogs, lightboxes, sheets, and property dialogs.
- Some selectors (`.property-dialog-*`, `.account-page`) assume matching markup elsewhere.

`index.html` — the HTML shell

```
<title>RewaCiti</title>
<script>
  (function(w,d,s,r,n){w.TrustpilotObject=n;w[n]=w[n]||function(){(w[n].q=w[n].q||[]).push(arguments)};
      a=d.createElement(s);a.async=1;a.src=r;a.type='text/java'+s;f=d.getElementsByTagName(s)[0];
      f.parentNode.insertBefore(a,f)})(window,document,'script', 'https://invitejs.trustpilot.com/tp.min.js', 'tp');
      tp('register', 'V6p7jWrntHaozFCu');
</script>
</head>
<body class ="bg-[#1E1E1E] text-white">
  <div id="root"></div>
  <script type="module" src="/src/app/main.tsx"></script>
</body>
```

How it works

- Loads DM Sans and DM Serif Display from Google Fonts.
- Sets SEO/social meta tags and the Trustpilot bootstrap.
- Mounts the app at `#root`.
- The body baseline is dark (`bg-[#1E1E1E] text-white`), which the theme layer can override.

`vercel.json` — SPA routing

```
{
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

How it works

- Serves `index.html` for unknown paths so client-side routing works on refresh and direct links.

`.env.example` — environment variables

```
# SabiFlow's API
VITE_API_URL=https://api.sabiflow.com/api

# RewaCiti's company id, for customer sign-in (/customers/*)
VITE_COMPANY_ID=
# "Continue with Google" needs no key. In SabiFlow, Customers → Settings →
# "Google sign-in on your own website or app" must list this site's
# /auth/google address exactly: https://rewaciti.com/auth/google (and
# http://localhost:5173/auth/google while building).

# SabiFlow → Real Estate → API: a public key (pk_live_…) with the website's
# addresses as its allowed websites. A pk_test_ key checks every request the
# same way but saves nothing, for trying the site out.
VITE_SABIFLOW_PUBLIC_KEY=
```

How it works

- `VITE_API_URL` — SabiFlow API base.
- `VITE_COMPANY_ID` — RewaCiti's tenant ID for `/customers/*`.
- `VITE_SABIFLOW_PUBLIC_KEY` — public Real Estate key (`pk_…`) restricted by allowed website addresses.
- All `VITE_*` values are bundled into the browser; never put a secret (`sk_…`) key here.
- Google sign-in also requires the exact callback URL to be registered in SabiFlow.

`tailwind.config.ts` — Tailwind (legacy config)

```
export default {
  darkMode: "class",
```

```
  content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}"],
  theme: {
    extend: {},
  },
  plugins: [],
} satisfies Config;
```

How it works

- Enables class-based dark mode and scans the app files.
- The project primarily uses Tailwind v4 CSS directives in `App.css`.

`components.json` — shadcn-style tooling

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "new-york",
  "rsc": false,
  "tsx": true,
  "tailwind": {
    "config": "",
    "css": "src/App.css",
    "baseColor": "neutral",
    "cssVariables": true,
    "prefix": ""
  },
  "iconLibrary": "lucide",
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils",
    "ui": "@/components/ui",
    "lib": "@/lib",
    "hooks": "@/hooks"
  },
  "registries": {}
}
```

How it works

- Guides component-generator conventions and import aliases.
- Note that the `@/` aliases must be provided by the tooling/tsconfig setup.

---

15. End-to-end data flows

Every backend endpoint the app uses

Customer API (`src/features/auth/services/authAPI.ts`)

Method	Endpoint	Used by
POST	`/customers/login`	Login
POST	`/customers/google/sign-in`	GoogleCallback
POST	`/customers/register`	Login (sign-up)
POST	`/customers/verify-email`	VerifyEmail
POST	`/customers/resend-verification-otp`	VerifyEmail
POST	`/customers/forgot-password`	ForgotPassword
POST	`/customers/reset-password`	ResetPassword
GET	`/customers/profile`	Profile, AccountPage, prefills
PUT	`/customers/profile`	Profile
PUT	`/customers/change-password`	Profile, AccountPage

Real Estate API (`realEstateApi`, base `/real-estate/public/v1`)

Method	Endpoint	Used by
GET	`/properties`	`propertiesAPI.list`
GET	`/properties/{idOrSlug}`	`propertiesAPI.get`
GET	`/properties/{idOrSlug}/related`	`propertiesAPI.related`
GET	`/properties/filters`	`propertiesAPI.filters`
GET	`/locations`	`propertiesAPI.locations`
GET	`/property-types`	`propertiesAPI.types`
GET	`/forms`	`propertiesAPI.forms`
GET	`/me/saved-properties`	wishlist / account
PUT	`/me/saved-properties/{id}`	save
DELETE	`/me/saved-properties/{id}`	unsave
POST	`/properties/{idOrSlug}/purchase`	payment modal
POST	`/inquiries`	inquiry modal
POST	`/contact`	contact page
POST	`/property-requests`	request page
POST	`/co-rent-requests`	co-rent page
POST	`/agent-applications`	list-property page
POST	`/agent-reports`	report modal
POST	`/ratings`	rating modal
GET	`/properties/{propertyId}/slots`	inspection modal
POST	`/bookings`	inspection modal
GET	`/me/bookings`	account visits
POST	`/me/bookings/{id}/reschedule`	account visits

Method	Endpoint	Used by
POST	`/me/bookings/{id}/cancel`	account visits
GET	`/me/inquiries`	account messages
GET	`/me/requests`	account messages
GET	`/me/deals`	account payments
POST	`/payments/start`	`payInvoice`
POST	`/payments/verify`	`payInvoice`

Other

Method	Endpoint	Used by
POST	`https://api.sabiflow.com/api/crm/deals/grestor`	Footer newsletter
GET	`/data/Blog.json`	blog store
GET	`/data/Comments.json`	comment store
GET	`/data/FAQ.json`	FAQ store

App boot

1. `index.html` loads `/src/app/main.tsx`.
2. `main.tsx` imports `App.css` and Leaflet's CSS, then renders `<App />` in `StrictMode`.
3. `App.tsx` creates the router, applies the stored theme to `<html>`, and hydrates the wishlist when a session exists.
4. Pages mount and read their data from Zustand stores or static JSON.

Auth

1. **Sign-up:** `Login` → `authAPI.signUp` → `sessionStorage["pendingVerification"]` → `VerifyEmail` → `authAPI.verifyEmail` → store token + customer → `/`.
2. **Sign-in:** `Login` → `authAPI.login` → store token + customer → `fetchWishlist()` → `/`.
3. **Google:** `Login` → `startGoogleSignIn` (PKCE) → `/auth/google` → `authAPI.googleSignIn` → store session → `next`.
4. **Forgot password:** `ForgotPassword` → `authAPI.forgotPassword` → OTP stored in `sessionStorage` → `ResetPassword` → `authAPI.resetPassword` → `/auth/login`.
5. **Logout:** `useAuthStore.logout()` clears cookies and state; navigation goes to `/`.

Browsing properties

1. `PropertiesPage` reads filters from state/URL and calls `fetchProperties(1, filters)`.
2. `usePropertyStore` calls `propertiesAPI.list`, which hits `GET /properties`.
3. Each `PublicListing` is mapped by `toProperty` into a `Property`.
4. Cards render; list/map modes share the same `properties` array.
5. Pagination calls `fetchProperties(nextPage, filters)` and updates both `page` and `apiPage`.

Property detail

1. `PropertyDetailsPage` calls `fetchPropertyBySlug(slug)`.
2. The store serves a cached copy immediately, then calls `propertiesAPI.get(slug)` and `fetchForms()` in parallel.
3. The fresh listing is mapped with `toProperty(listing, forms.property)` and upserted into the list cache.
4. Related listings are loaded separately with `fetchRelatedProperties`.
5. Booking, payment, inquiry, report, and rating actions open their respective modals.

Wishlist / shortlist

1. On login and app boot, `fetchWishlist()` calls `GET /me/saved-properties`.
2. `toggleShortlist(property)` updates the UI optimistically.
3. It calls `PUT` or `DELETE /me/saved-properties/{id}`.
4. The server's returned saved list replaces local state; on error, the previous list is restored and a toast is shown.

Buying a property

1. `PropertyPaymentModal` calls `propertiesAPI.purchase(id, contact, key)`.
2. SabiFlow creates a deal and holds the property, returning a `Deal`, an optional `bankTransfer` block, and payment lines.
3. Above the online limit, the modal shows bank-transfer details.
4. Otherwise it calls `payInvoice` → `POST /payments/start`.
5. Paystack opens in a popup; on success `POST /payments/verify` confirms.
6. The property is refetched and, if enabled, the rating modal opens.

Booking a visit

1. `BookInspectionModal` loads `GET /properties/{id}/slots?date=...`.
2. It submits `POST /bookings` with an idempotency key.
3. If a fee is due, the booking is held and `payInvoice` is called with the invoice ID.
4. The account page lists visits with `GET /me/bookings` and can reschedule or cancel.

Lead capture (CRM)

- Inquiry, contact, property request, co-rent, agent application, and report forms all post through `requestsAPI`.
- Each uses an `Idempotency-Key` from `useRequestKey`, so a double submit cannot create duplicate records.
- The footer newsletter posts directly to SabiFlow's CRM guest-deals endpoint.

Static content

- Blog, comments, and FAQ stores load JSON from `public/data`.
- There is no API call, error state, or server pagination for these; pagination is client-side.

Where state lives

State	Store / location	Persistence
Auth session	`useAuthStore`	`localStorage["auth-storage"]` + cookies
Theme	`useThemeStore`	`localStorage["theme"]`
Listings, wishlist, forms, locations	`usePropertyStore`	In memory (promise caches)
Blog posts	`useBlogStore`	In memory
Comments	`useCommentStore`	In memory
FAQs	`useFAQStore`	In memory
Form drafts	Cookies per form	`rewaciti_*` cookies
Pending sign-up / reset	`sessionStorage`	Cleared after use
Google PKCE verifier	`sessionStorage`	Single-use
UI state (modals, tabs, filters)	Component `useState`	None

End of walkthrough.